

SOLIDWORKS DimXpert API (.NET) — Examples (2/7): Get_and_Set_Location_Dimension_Example_VB- Get_DimXpert_Datum_Example_VB

SOLIDWORKS API Documentation (converted from CHM)

Get_and_Set_Location_Dimension_Example_VBNET (Topic)

SOLIDWORKS API Help

Get and Set Location Dimension Example (VB.NET)

This example shows how to get and set DimXpert distance-between or angle-between dimensions.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\dimxpert\bracket_ads_plusminus.sldprt.  
' 2. Multi-select two faces that:  
'    a. share a common edge to get the angle-between dimension.  
'    b. do not share a common edge to get the distance-between dimension.  
' 3. Open an Immediate window.  
' 4. Ensure that the latest SOLIDWORKS DimXpert interop assembly is referenced:  
'    a. Right-click the project in Project Explorer.  
'    b. Select Add Reference.  
'    c. Click the Browse tab.  
'    d. Find and select install_dir\api\redist\swdimxpert.dll.  
'  
' Postconditions:  
' 1. Inspect the Immediate window.  
' 2. Observe the DistanceBetween1 or AngleBetween1 annotation on the  
'    DimXpertManager tab.  
'  
' NOTE: Because this part is used elsewhere, do not save changes.  
'-----  
  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports SolidWorks.Interop.swdimxpert  
Imports System  
Imports System.Diagnostics  
  
Partial Class SolidWorksMacro  
  
    Dim dimXpertPart As dimXpertPart  
  
    Sub Main()  
  
        Dim swModelDoc As ModelDoc2  
        swModelDoc = swapp.ActiveDoc  
        Dim annoType As Long  
  
        If swModelDoc Is Nothing Then  
            Exit Sub  
        End If  
  
        Dim dimXpertMgr As DimXpertManager  
        dimXpertMgr =  
swapp.IActiveDoc2.Extension.DimXpertManager(swapp.IActiveDoc2.IGetActiveConfiguration().Name,  
True)  
        Debug.Print("Model: " & swapp.IActiveDoc2.GetPathName)  
  
        Dim dimXpertPartObj As dimXpertPart  
        dimXpertPartObj = dimXpertMgr.dimXpertPart  
        dimXpertPart = dimXpertPartObj  
  
        Dim dimOption As DimXpertDimensionOption  
        dimOption = dimXpertPart.GetDimOption  
  
        ' DimensionPositionOption works only for linear location dimensions  
        dimOption.DimensionPositionOption = 0 ' Specify a dimension along the normal to the face
```

```

' Insert the dimension
dimXpertPart.InsertLocationDimension(dimOption)

Dim vAnnotations As Object
vAnnotations = dimXpertPart.GetAnnotations()

Debug.Print("-----")
Debug.Print("Annotations...")
Debug.Print("-----")

Dim annotationTemp As DimXpertAnnotation
Dim annotationIndex As Long
For annotationIndex = 0 To UBound(vAnnotations)
    annotationTemp = vAnnotations(annotationIndex)
    Call GeneralInfo(annotationTemp)
    annoType = annotationTemp.Type
    Call DimensionToleranceData(annotationTemp)
Next

End Sub

Private Sub GeneralInfo(ByVal annotation As DimXpertAnnotation)

    Dim annoType As String
    Dim modelObj As Object
    Dim modelFeature As Feature

    Debug.Print("")
    Debug.Print("Name: " + annotation.Name)

    annoType = annotationTypeNameFromObject(annotation)

    Debug.Print("Type: " + annoType)
    Debug.Print("Display Entity: " + DisplayEntity(annotation))

    modelObj = annotation.GetModelFeature
    modelFeature = modelObj
    If Not (modelFeature Is Nothing) Then
        Debug.Print("ModelFeature: " + modelFeature.Name + " (" + modelFeature.GetTypeName2()
+ ")")
    End If

End Sub

Private Sub DimensionToleranceData(ByVal annotation As DimXpertDimensionTolerance)

    Dim isAngleType As Boolean
    Dim annoType As Long
    Dim upper As Double, lower As Double
    Dim plus As Double, minus As Double
    Dim boolstatus As Boolean

    annoType = annotation.Type
    isAngleType = False

    Debug.Print("")
    Debug.Print("Dimension Tolerance Compartment")

    If annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_DistanceBetween Then
        Dim distancebetween As DimXpertDistanceBetweenDimTol
        distancebetween = annotation
        Call DistanceBetweenData(distancebetween)

    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_AngleBetween Then
        isAngleType = True
        Dim angleBetween As IDimXpertAngleBetweenDimTol
        angleBetween = annotation
    End If

```

```

' the origin and tolerance feature
Debug.Print("Origin Feature: " + angleBetween.OriginFeature.Name)

' is supplement angle
Debug.Print("Supplement Angle: " + IIf(angleBetween.Supplement, "True", "False"))

End If

' conversion for radians to degrees when dimension type is angle

Dim dbl As Double
If isAngleType Then
    dbl = 57.2957795130823
Else
    dbl = 1.0#
End If

' the nominal, and upper and lower limits of size of the dimension

Debug.Print("")
Debug.Print("Compute Nominal Dimension: " + FormatDouble(annotation.GetNominalValue *
dbl))
boolstatus = annotation.GetUpperAndLowerLimit(upper, lower)
Debug.Print("Get Upper Limit: " + FormatDouble(upper * dbl))
Debug.Print("Get Lower Limit: " + FormatDouble(lower * dbl))

' the upper and lower tolerance value by type
Select Case annotation.DimensionType

    Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_BlockTolerance
        Debug.Print("Dimension Type: Block Tolerance")

        ' block tolerance

    Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_BlockToleranceNoNominal
        Dim blockTols As DimXpertBlockTolerances
        blockTols = dimXpertPart.GetBlockTolerances

        Select Case blockTols.Type

            Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ASMEInch
                Debug.Print("Dimension Type: Block Tolerance No Nominal")
                If isAngleType Then
                    Debug.Print("Angular Block Tolerance")
                Else
                    Debug.Print("Block Tolerance Decimal Places: " +
Format(annotation.BlockToleranceDecimalPlaces, "##,##0"))
                End If

            Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ISO2768
                Debug.Print("Dimension Type: General Tolerance")

        End Select

    Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_ISOLimitsAndFits
        Debug.Print("Dimension Type: Limits and Fits")

        ' limits and fits tolerance
    Case
swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_ISOLimitsAndFitsNoNominal

        Debug.Print("Dimension Type: Limits and Fits No Nominal")
        Debug.Print("Limits and Fits: " + annotation.LimitsAndFitsCode)

        ' limit dimension
    Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_LimitDimension

        Debug.Print("Dimension Type: Limit Dimension")
        boolstatus = annotation.GetUpperAndLowerLimit(upper, lower)
        Debug.Print("Get Upper Limit: " + FormatDouble(upper * dbl))

```

```

        Debug.Print("Get Lower Limit: " + FormatDouble(lower * dbl))

    Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_MAXTolerance
        Debug.Print("Dimension Type: MAXTolerance")

    Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_MINTolerance
        Debug.Print("Dimension Type: MINTolerance")

    Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_NoTolerance
        Debug.Print("Dimension Type: NoTolerance")

    Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_PlusMinusDimension
        Debug.Print("Dimension Type: Plus Minus Dimension")

        ' plus and minus dimension

    Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_PlusMinusNoNominal
        Debug.Print("Dimension Type: Plus Minus No Nominal")
        boolstatus = annotation.GetPlusAndMinusTolerance(plus, minus)
        Debug.Print("Plus Tolerance: " + FormatDouble(plus * dbl))
        Debug.Print("Minus Tolerance: " + FormatDouble(minus * dbl))

End Select

End Sub

Private Sub DistanceBetweenData(ByVal annotation As DimXpertDistanceBetweenDimTol)

    Dim feature As DimXpertFeature
    Dim featureFosUsage As Long
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean

    feature = Nothing
    ' the origin and tolerance feature along with their feature of size usage (min, max,
center)
    boolstatus = annotation.GetOriginFeature(feature, featureFosUsage)

    Debug.Print("")
    Debug.Print("Origin Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))
    boolstatus = annotation.GetFeature(feature, featureFosUsage)
    Debug.Print("Tolerance Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))

    ' The direction vector

    boolstatus = annotation.GetDirectionVector(I, J, K)
    Debug.Print("")
    Debug.Print("Direction Vector: " + FormatVector(I, J, K))

End Sub

Private Function annotationTypeNameFromObject(ByVal anno As DimXpertAnnotation) As String
    annotationTypeNameFromObject = annotationTypeNameFromTypeNumber(anno.Type)
End Function

Private Function annotationTypeNameFromTypeNumber(ByVal annoTypeIndex As Long) As String

    Select Case annoTypeIndex

        Case swDimXpertAnnotationType_e.swDimXpertDimTol_DistanceBetween
            annotationTypeNameFromTypeNumber = "DistanceBetween Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_AngleBetween
            annotationTypeNameFromTypeNumber = "AngleBetween Dim"
        Case Else
            annotationTypeNameFromTypeNumber = "Unknown"

    End Select

End Function

End Function

```

```

Private Function DisplayEntity(ByVal annotation As DimXpertAnnotation) As String

    Dim str As String
    Dim dispEnt As Object
    Dim swAnnot As Annotation

    dispEnt = annotation.GetDisplayEntity

    str = Nothing
    If Not dispEnt Is Nothing Then
        If TypeOf dispEnt Is Annotation Then
            swAnnot = dispEnt
            str = swAnnot.GetName
        End If
    End If

    DisplayEntity = str

End Function

Private Function FosUsage(ByVal value As Long) As String

    Dim str As String
    Select Case value

        Case swDimXpertDistanceFosUsage_e.swDimXpertDistanceFosUsage_Center
            str = "Center"

        Case swDimXpertDistanceFosUsage_e.swDimXpertDistanceFosUsage_MaximumSide
            str = "Max"

        Case swDimXpertDistanceFosUsage_e.swDimXpertDistanceFosUsage_MinimumSide
            str = "Min"

        Case Else
            str = "N/A"

    End Select

    FosUsage = str

End Function

Private Function FormatVector(ByVal I As Double, ByVal J As Double, ByVal K As Double) As String

    Dim str As String
    str = FormatDouble(I) + ", " + FormatDouble(J) + ", " + FormatDouble(K)
    FormatVector = str

End Function

Private Function FormatDouble(ByVal value As Double) As String

    Dim str As String
    str = Format(value, "##,##0.0000")
    FormatDouble = str

End Function

Private Function RadiansToDegrees(ByVal value As Double) As String

    Dim str As String
    str = Format((value * 57.2957795130823), "##,##0.00")
    RadiansToDegrees = str

End Function

Public swApp As SldWorks

```

End Class

Get and Set Pattern Example (C#)

This example shows how to get and set DimXpert pattern features.

```
//-----  
// Preconditions:  
// 1. Open public_documents\samples\tutorial\dimxpert\bracket_auto_manual.sldprt.  
// 2. Select one of the CBORE hole faces in the SOLIDWORKS viewer.  
// 3. Open an Immediate window.  
// 4. Ensure that the latest SOLIDWORKS DimXpert interop assembly is referenced:  
//    a. Right-click the project in Project Explorer.  
//    b. Select Add Reference.  
//    c. Click the Browse tab.  
//    d. Find and select install_dir\api\redist\swdimxpert.dll.  
//  
// Postconditions: Inspect the Immediate Window and the DimXpertManager tab.  
//  
// NOTE: Because this part is used elsewhere, do not save changes.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using SolidWorks.Interop.swdimxpert;  
using System;  
using System.Diagnostics;  
  
namespace InsertPattern_CSharp.csproj  
{  
  
    partial class SolidWorksMacro  
    {  
        DimXpertPart dimXpertPart;  
        ModelDoc2 swModelDoc;  
        swDimXpertFeatureType_e featureType;  
        string msgStr;  
        string msgStr2;  
        string msgStr3;  
        bool retval;  
        int n;  
  
        public void Main()  
        {  
  
            swModelDoc = (ModelDoc2)swApp.ActiveDoc;  
  
            if (swModelDoc == null)  
            {  
                return;  
            }  
  
            DimXpertManager dimXpertMgr = default(DimXpertManager);  
            dimXpertMgr =  
swApp.IActiveDoc2.Extension.get_DimXpertManager(swApp.IActiveDoc2.IGetActiveConfiguration().Name,  
true);  
  
            Debug.Print("Model: " + swApp.IActiveDoc2.GetPathName());  
  
            DimXpertPart dimXpertPartObj = default(DimXpertPart);  
            dimXpertPartObj = (DimXpertPart)dimXpertMgr.DimXpertPart;  
            dimXpertPart = dimXpertPartObj;  
  
            SelectionMgr selectMgr = default(SelectionMgr);  
            selectMgr = (SelectionMgr)swApp.IActiveDoc2.SelectionManager;
```

```

DimXpertDimensionOption dimOption = default(DimXpertDimensionOption);
dimOption = dimXpertPart.GetDimOption();

int patternType = 0;
patternType = 2;
// collection pattern

bool findall = false;
findall = true;
// find all similar features on this face

int[] dimarray = new int[1];
dimarray[0] = -1;
// default
object dimvar = null;
dimvar = dimarray;
dimOption.FeatureSelectorOptions = dimvar;

retval = selectMgr.SetSelectedObjectMark(1, 51,
(int)swSelectionMarkAction_e.swSelectionMarkSet);

// Insert the collection pattern feature
retval = dimXpertPart.InsertPattern(dimOption, patternType, findall);

int featCount = 0;
featCount = dimXpertPart.GetFeatureCount();

if (!(featCount == 0))
{
    IDimXpertFeature patternFeature = default(IDimXpertFeature);
    patternFeature = (DimXpertFeature)dimXpertPart.GetFeature("Collection1");

    msgStr = patternFeature.Name + " is a DimXpert collection pattern feature.";
    Debug.Print("");
    Debug.Print(msgStr);

    featureType = ((IDimXpertPatternFeature)patternFeature).PatternType;

    GetPatternType(ref featureType, ref msgStr2);

    int featureCount = 0;

    featureCount = ((IDimXpertPatternFeature)patternFeature).GetSubFeatureCount();
    msgStr = " Number of ";
    msgStr3 = featureCount.ToString();
    Debug.Print(msgStr + msgStr2 + " sub-features in this pattern is " + msgStr3);

    object[] subfeatures = null;
    subfeatures = (object[])
((IDimXpertPatternFeature)patternFeature).GetSubFeatures();

    Debug.Print(" Sub-features of Collection1:");
    DimXpertFeature subFeature = default(DimXpertFeature);

    for (n = 0; n <= subfeatures.GetUpperBound(0); n++)
    {
        subFeature = (DimXpertFeature)subfeatures[n];
        Debug.Print(" " + subFeature.Name);
    }
}
else
{
    Debug.Print("Please select a CBORE hole face in the viewer and run this macro
again.");
}
}

```

```

public void GetPatternType(ref swDimXpertFeatureType_e featureType, ref string msgStr2)
{
    if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Plane))
    {
        msgStr2 = "Plane";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Cylinder))
    {
        msgStr2 = "Cylinder";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Cone))
    {
        msgStr2 = "Cone";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Extrude))
    {
        msgStr2 = "Extrude";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Fillet))
    {
        msgStr2 = "Fillet";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Chamfer))
    {
        msgStr2 = "Chamfer";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_CompoundHole))
    {
        msgStr2 = "CompoundHole";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_CompoundWidth))
    {
        msgStr2 = "CompoundWidth";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_CompoundNotch))
    {
        msgStr2 = "CompoundNotch";
    }
    else if ((featureType ==
swDimXpertFeatureType_e.swDimXpertFeature_CompoundClosedSlot3D))
    {
        msgStr2 = "CompoundClosedSlot3D";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_IntersectPoint))
    {
        msgStr2 = "IntersectPoint";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_IntersectLine))
    {
        msgStr2 = "IntersectLine";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_IntersectCircle))
    {
        msgStr2 = "IntersectCircle";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_IntersectPlane))
    {
        msgStr2 = "IntersectPlane";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Pattern))
    {
        msgStr2 = "Pattern";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Sphere))
    {
        msgStr2 = "Sphere";
    }
}

```

```
        else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_BestfitPlane))
        {
            msgStr2 = "Bestfit plane";
        }
        else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Surface))
        {
            msgStr2 = "Surface";
        }

    }

    public SldWorks swApp;
}
}
```

Get and Set Pattern Example (VBA)

This example shows how to get and set DimXpert pattern features.

```
' -----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\dimxpert\bracket_auto_manual.sldprt.  
' 2. Select one of the CBORE hole faces in the SOLIDWORKS viewer.  
' 3. Open an Immediate window .  
' 4. Ensure that the latest SOLIDWORKS DimXpert type library is loaded:  
'     a. Select Tools > References.  
'     b. Click Browse.  
'     c. Find and select install_dir\swdimxpert.tlb.  
'  
' Postconditions: Inspect the Immediate Window and the DimXpertManager tab.  
'  
' NOTE: Because this part is used elsewhere, do not save changes.  
' -----
```

Option Explicit

Dim dimXpertPart As dimXpertPart

Dim swApp As SldWorks.SldWorks

```
Dim swModelDoc As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swConfig As SldWorks.Configuration  
Dim swFeature As SldWorks.feature  
Dim swAnn As SldWorks.feature  
Dim swSchema As SldWorks.DimXpertManager  
Dim swDXPart As dimXpertPart  
Dim schemeOption As DimXpertAutoDimSchemeOption  
Dim featureType As swDimXpertFeatureType_e  
Dim features As Variant  
Dim appliedFeatures As Variant  
Dim appliedAnnotations As Variant  
Dim appliedAnnotation As DimXpertAnnotation  
Dim feature As DimXpertFeature  
Dim appliedFeature As DimXpertFeature  
Dim msgStr As String  
Dim msgStr2 As String  
Dim msgStr3 As String  
Dim msgStr4 As String  
Dim n As Long  
Dim o As Long  
Dim p As Long
```

Dim boolstatus As Boolean

Dim annoType As Long

Dim retval As Boolean

Sub Main()

Set swApp = Application.SldWorks

Set swModelDoc = swApp.ActiveDoc

If swModelDoc Is Nothing Then

Exit Sub

End If

Dim dimXpertMgr As SldWorks.DimXpertManager

Set dimXpertMgr =

swApp.IActiveDoc2.Extension.DimXpertManager(swApp.IActiveDoc2.IGetActiveConfiguration().Name,

```

True)
    Debug.Print "Model: " & swApp.IActiveDoc2.GetPathName

    Dim dimXpertPartObj As dimXpertPart
    Set dimXpertPartObj = dimXpertMgr.dimXpertPart
    Set dimXpertPart = dimXpertPartObj

    Dim selectMgr As SelectionMgr
    Set selectMgr = swApp.IActiveDoc2.SelectionManager

    Dim dimOption As DimXpertDimensionOption
    Set dimOption = dimXpertPart.GetDimOption

    Dim patternType As Long
    patternType = 2 ' collection pattern

    Dim findall As Boolean
    findall = True ' find all similar features on this face

    Dim dimarray(0) As Long
    dimarray(0) = -1 ' default
    Dim dimvar As Variant
    dimvar = dimarray
    dimOption.FeatureSelectorOptions = dimvar

    'Mark the current selection with an index value greater than 50
    retval = selectMgr.SetSelectedObjectMark(1, 51, swSelectionMarkSet)

    ' Insert the collection pattern feature
    retval = dimXpertPart.InsertPattern(dimOption, patternType, findall)

    Dim featCount As Long
    featCount = dimXpertPart.GetFeatureCount

    If Not (featCount = 0) Then
        Dim patternFeature As IDimXpertPatternFeature
        Set patternFeature = dimXpertPart.GetFeature("Collection1")
        msgStr = patternFeature.Name + " is a DimXpert collection pattern feature."
        Debug.Print ""
        Debug.Print msgStr

        featureType = patternFeature.patternType
        Call GetPatternType(featureType, msgStr2)
        Dim featureCount As Integer
        featureCount = patternFeature.GetSubFeatureCount()
        msgStr = "      Number of "
        msgStr3 = featureCount
        Debug.Print msgStr + msgStr2 + " sub-features in this pattern is " + msgStr3

        Dim subfeatures As Variant
        subfeatures = patternFeature.GetSubFeatures()
        Debug.Print "      Sub-features of Collection1:"
        Dim subFeature As DimXpertCompoundHoleFeature

        For n = 0 To UBound(subfeatures)
            Set subFeature = subfeatures(n)
            Debug.Print "      " + subFeature.Name
        Next

    Else
        Debug.Print "Please select a CBORE hole face in the viewer and run this macro again."
    End If

```

End If

End Sub

```
Public Sub GetPatternType(ByRef featureType, ByRef msgStr2)
    If (featureType = swDimXpertFeature_Plane) Then
        msgStr2 = "Plane"
    ElseIf (featureType = swDimXpertFeature_Cylinder) Then
        msgStr2 = "Cylinder"
    ElseIf (featureType = swDimXpertFeature_Cone) Then
        msgStr2 = "Cone"
    ElseIf (featureType = swDimXpertFeature_Extrude) Then
        msgStr2 = "Extrude"
    ElseIf (featureType = swDimXpertFeature_Fillet) Then
        msgStr2 = "Fillet"
    ElseIf (featureType = swDimXpertFeature_Chamfer) Then
        msgStr2 = "Chamfer"
    ElseIf (featureType = swDimXpertFeature_CompoundHole) Then
        msgStr2 = "CompoundHole"
    ElseIf (featureType = swDimXpertFeature_CompoundWidth) Then
        msgStr2 = "CompoundWidth"
    ElseIf (featureType = swDimXpertFeature_CompoundNotch) Then
        msgStr2 = "CompoundNotch"
    ElseIf (featureType = swDimXpertFeature_CompoundClosedSlot3D) Then
        msgStr2 = "CompoundClosedSlot3D"
    ElseIf (featureType = swDimXpertFeature_IntersectPoint) Then
        msgStr2 = "IntersectPoint"
    ElseIf (featureType = swDimXpertFeature_IntersectLine) Then
        msgStr2 = "IntersectLine"
    ElseIf (featureType = swDimXpertFeature_IntersectCircle) Then
        msgStr2 = "IntersectCircle"
    ElseIf (featureType = swDimXpertFeature_IntersectPlane) Then
        msgStr2 = "IntersectPlane"
    ElseIf (featureType = swDimXpertFeature_Pattern) Then
        msgStr2 = "Pattern"
    ElseIf (featureType = swDimXpertFeature_Sphere) Then
        msgStr2 = "Sphere"
    ElseIf (featureType = swDimXpertFeature_BestfitPlane) Then
        msgStr2 = "Bestfit plane"
    ElseIf (featureType = swDimXpertFeature_Surface) Then
        msgStr2 = "Surface"

    End If
End Sub
```

Get and Set Pattern Example (VB.NET)

This example shows how to get and set DimXpert pattern features.

```
' -----  
' Preconditions:  
' 1. Open: public_documents\samples\tutorial\dimxpert\bracket_auto_manual.sldprt.  
' 2. Select one of the CBORE hole faces in the SOLIDWORKS viewer.  
' 3. Open an Immediate window.  
' 4. Ensure that the latest SOLIDWORKS DimXpert interop assembly is referenced:  
'     a. Right-click the project in Project Explorer.  
'     b. Select Add Reference.  
'     c. Click the Browse tab.  
'     d. Find and select install_dir\api\redist\swdimxpert.dll.  
  
'  
' Postconditions: Inspect the Immediate window and the DimXpertManager tab.  
'  
' NOTE: Because this part is used elsewhere, do not save changes.  
' -----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports SolidWorks.Interop.swdimxpert  
Imports System  
Imports System.Diagnostics  
  
Partial Class SolidWorksMacro  
  
    Dim dimXpertPart As dimXpertPart  
    Dim swModelDoc As ModelDoc2  
    Dim featureType As swDimXpertFeatureType_e  
    Dim msgStr As String  
    Dim msgStr2 As String  
    Dim msgStr3 As String  
    Dim n As Long  
    Dim retval As Boolean  
  
    Sub Main()  
  
        swModelDoc = swApp.ActiveDoc  
  
        If swModelDoc Is Nothing Then  
            Exit Sub  
        End If  
  
        Dim dimXpertMgr As DimXpertManager  
        dimXpertMgr =  
swApp.IActiveDoc2.Extension.DimXpertManager(swApp.IActiveDoc2.IGetActiveConfiguration().Name,  
True)  
        Debug.Print("Model: " & swApp.IActiveDoc2.GetPathName)  
  
        Dim dimXpertPartObj As dimXpertPart  
        dimXpertPartObj = dimXpertMgr.dimXpertPart  
        dimXpertPart = dimXpertPartObj  
  
        Dim selectMgr As SelectionMgr  
        selectMgr = swApp.IActiveDoc2.SelectionManager  
  
        Dim dimOption As DimXpertDimensionOption  
        dimOption = dimXpertPart.GetDimOption  
  
        Dim patternType As Long  
        patternType = 2 ' collection pattern
```

```

Dim findall As Boolean
findall = True ' find all similar features on this face

Dim dimarray(0) As Long
dimarray(0) = -1 ' default
Dim dimvar As Object
dimvar = dimarray
dimOption.FeatureSelectorOptions = dimvar

' Mark the current selection with an index value greater than 50
retval = selectMgr.SetSelectedObjectMark(1, 51,
swSelectionMarkAction_e.swSelectionMarkSet)

' Insert the collection pattern feature
retval = dimXpertPart.InsertPattern(dimOption, patternType, findall)

Dim featCount As Long
featCount = dimXpertPart.GetFeatureCount

If Not (featCount = 0) Then

    Dim patternFeature As IDimXpertPatternFeature
    patternFeature = dimXpertPart.GetFeature("Collection1")

    msgStr = patternFeature.Name + " is a DimXpert collection pattern feature."
    Debug.Print("")
    Debug.Print(msgStr)

    featureType = patternFeature.patternType

    Call GetPatternType(featureType, msgStr2)

    Dim featureCount As Integer

    featureCount = patternFeature.GetSubFeatureCount()
    msgStr = "    Number of "
    msgStr3 = featureCount
    Debug.Print(msgStr + msgStr2 + " sub-features in this pattern is " + msgStr3)

    Dim subfeatures As Object
    subfeatures = patternFeature.GetSubFeatures()

    Debug.Print("    Sub-features of Collection1:")
    Dim subFeature As IDimXpertCompoundHoleFeature

    For n = 0 To UBound(subfeatures)
        subFeature = subfeatures(n)
        Debug.Print("        " + subFeature.Name)
    Next

Else
    Debug.Print("Please select a CBORE hole face in the viewer and run this macro
again.")
End If

End Sub

```

```

Public Sub GetPatternType(ByRef featureType As swDimXpertFeatureType_e, ByRef msgStr2 As
String)

    If (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Plane) Then
        msgStr2 = "Plane"

    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Cylinder) Then
        msgStr2 = "Cylinder"
    End If
End Sub

```

```

ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Cone) Then
    msgStr2 = "Cone"

ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Extrude) Then
    msgStr2 = "Extrude"

ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Fillet) Then
    msgStr2 = "Fillet"

ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Chamfer) Then
    msgStr2 = "Chamfer"

ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundHole) Then
    msgStr2 = "CompoundHole"

ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundWidth) Then
    msgStr2 = "CompoundWidth"

ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundNotch) Then
    msgStr2 = "CompoundNotch"

ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundClosedSlot3D)
Then
    msgStr2 = "CompoundClosedSlot3D"

ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectPoint) Then
    msgStr2 = "IntersectPoint"

ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectLine) Then
    msgStr2 = "IntersectLine"

ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectCircle) Then
    msgStr2 = "IntersectCircle"

ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectPlane) Then
    msgStr2 = "IntersectPlane"

ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Pattern) Then
    msgStr2 = "Pattern"

ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Sphere) Then
    msgStr2 = "Sphere"

ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_BestfitPlane) Then
    msgStr2 = "Bestfit plane"

ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Surface) Then
    msgStr2 = "Surface"

End If

End Sub

Public swApp As SldWorks

End Class

```

Get and Set Size Dimension Example (C#)

This example shows how to get and set DimXpert compound width features.

```
//-----  
// Preconditions:  
// 1. Open public_documents\samples\tutorial\dimxpert\bracket_ads_plusminus.sldprt.  
// 2. Select an edge.  
// 3. Open the Immediate window.  
// 4. Ensure that the latest SOLIDWORKS DimXpert interop assembly is referenced:  
//    a. Right-click the project in Project Explorer.  
//    b. Select Add Reference.  
//    c. Click the Browse tab.  
//    d. Find and select install_dir\api\redist\swdimxpert.dll.  
//  
// Postconditions:  
// 1. Inspect the Immediate Window.  
// 2. Observe the Width1 feature and the Width2 size dimension on the  
//    DimXpertManager tab.  
//  
// NOTE: Because this part is elsewhere, do not save changes.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using SolidWorks.Interop.swdimxpert;  
using System;  
using System.Diagnostics;  
namespace InsertSize_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        DimXpertPart dimXpertPart;  
  
        public void Main()  
        {  
            ModelDoc2 swModelDoc = default(ModelDoc2);  
            swModelDoc = (ModelDoc2)swApp.ActiveDoc;  
            Feature swFeature = default(Feature);  
            swDimXpertFeatureType_e featureType = default(swDimXpertFeatureType_e);  
            object[] features = null;  
            object[] appliedFeatures = null;  
            object[] appliedAnnotations = null;  
            DimXpertAnnotation appliedAnnotation = default(DimXpertAnnotation);  
            DimXpertFeature feature = default(DimXpertFeature);  
            DimXpertFeature appliedFeature = default(DimXpertFeature);  
            string msgStr = null;  
            string msgStr2 = null;  
            string msgStr3 = null;  
            string msgStr4 = null;  
            int n = 0;  
            int o = 0;  
            int p = 0;  
            bool boolstatus = false;  
  
            if (swModelDoc == null)  
            {  
                return;  
            }  
  
            DimXpertManager dimXpertMgr = default(DimXpertManager);  
            dimXpertMgr =
```

```

swApp.IActiveDoc2.Extension.get_DimXpertManager(swApp.IActiveDoc2.IGetActiveConfiguration().Name,
true);

    Debug.Print("Model: " + swApp.IActiveDoc2.GetPathName());

    DimXpertPart dimXpertPartObj = default(DimXpertPart);
    dimXpertPartObj = (DimXpertPart)dimXpertMgr.DimXpertPart;
    dimXpertPart = dimXpertPartObj;

    DimXpertDimensionOption dimOption = default(DimXpertDimensionOption);
    dimOption = dimXpertPart.GetDimOption();

    // Specify the position of the dimension annotation
    double[] textPos = new double[3];
    textPos[0] = 0.05;
    textPos[1] = -0.03;
    textPos[2] = -0.03;
    object posvar = null;
    posvar = textPos;
    dimOption.TextPosition = posvar;

    int[] dimarray = new int[1];
    dimarray[0] = -1;
    // default feature
    object dimvar = null;
    dimvar = dimarray;
    dimOption.FeatureSelectorOptions = dimvar;

    // Insert the size dimension
    dimXpertPart.InsertSizeDimension(dimOption);

    int featCount = 0;
    featCount = dimXpertPart.GetFeatureCount();

    msgStr = "Total of ";
    msgStr2 = featCount.ToString();
    msgStr = msgStr + msgStr2 + " DimXpert features";
    Debug.Print("");
    Debug.Print(msgStr);

    // Get IDimXpert features through IDimXpertPart
    features = (object[])dimXpertPart.GetFeatures();
    if ((features != null))
    {
        msgStr = " Features: ";
        Debug.Print("");
        Debug.Print(msgStr);

        for (n = 0; n <= features.GetUpperBound(0); n++)
        {
            feature = (DimXpertFeature)features[n];
            Debug.Print(" " + "Feature name: " + (feature.Name));
            featureType = feature.Type;
            GetPatternType(ref featureType, ref msgStr2);

            msgStr = " Feature type ";
            msgStr3 = " is suppressed on the DimXpertManager tab? ";
            msgStr4 = feature.IsSuppressed().ToString();
            Debug.Print(msgStr + msgStr2 + msgStr3 + msgStr4);

            msgStr = " " + "Model feature: ";

            swFeature = (Feature)feature.GetModelFeature();

            if ((swFeature != null))
            {
                msgStr2 = swFeature.GetTypeName2();
                Debug.Print(msgStr + msgStr2);
            }
        }
    }

```

```

msgStr = " " + "Number of SOLIDWORKS face entities in this feature: ";
msgStr2 = feature.GetFaceCount().ToString();
Debug.Print(msgStr + msgStr2);

msgStr = " " + "Number of applied features: ";
msgStr2 = feature.GetAppliedFeatureCount().ToString();
Debug.Print(msgStr + msgStr2);

appliedFeatures = (object[])feature.GetAppliedFeatures();

if ((appliedFeatures != null))
{
    for (o = 0; o <= appliedFeatures.GetUpperBound(0); o++)
    {
        appliedFeature = (DimXpertFeature)appliedFeatures[o];
        Debug.Print(" " + "Applied feature name: " + (appliedFeature.Name));
    }
}

msgStr = " " + "Number of applied annotations: ";
msgStr2 = feature.GetAppliedAnnotationCount().ToString();
Debug.Print(msgStr + msgStr2);

appliedAnnotations = (object[])feature.GetAppliedAnnotations();

if ((appliedAnnotations != null))
{
    for (p = 0; p <= appliedAnnotations.GetUpperBound(0); p++)
    {
        appliedAnnotation = (DimXpertAnnotation)appliedAnnotations[p];
        Debug.Print(" " + "Applied annotation name: " +
(appliedAnnotation.Name));
    }
    Debug.Print(" ");
}

// Get specific information about the Width1 feature

IDimXpertCompoundWidthFeature widthFeature =
default(IDimXpertCompoundWidthFeature);
widthFeature = (IDimXpertCompoundWidthFeature)dimXpertPart.GetFeature("Width1");
msgStr = ((IDimXpertFeature)widthFeature).Name + " is a DimXpert compound width
feature.";
Debug.Print("");
Debug.Print(msgStr);
Debug.Print("");

// Get the nominal width coordinates

double width = 0;
double x = 0;
double y = 0;
double z = 0;
double i = 0;
double j = 0;
double k = 0;

Debug.Print("Nominal width of Width1");
Debug.Print("");
boolstatus = widthFeature.GetNominalCompoundWidth(ref width, ref x, ref y, ref z,
ref i, ref j, ref k);

msgStr = "Width is ";
msgStr2 = width.ToString();
Debug.Print(msgStr + msgStr2);

msgStr = "X-coordinate is ";
msgStr2 = x.ToString();
Debug.Print(msgStr + msgStr2);

```

```

        msgStr = "Y-coordinate is ";
        msgStr2 = y.ToString();
        Debug.Print(msgStr + msgStr2);

        msgStr = "Z-coordinate is ";
        msgStr2 = z.ToString();
        Debug.Print(msgStr + msgStr2);

        msgStr = "I-component of pierce vector is ";
        msgStr2 = i.ToString();
        Debug.Print(msgStr + msgStr2);

        msgStr = "J-component of pierce vector is ";
        msgStr2 = j.ToString();
        Debug.Print(msgStr + msgStr2);

        msgStr = "K-component of pierce vector is ";
        msgStr2 = k.ToString();
        Debug.Print(msgStr + msgStr2);
        Debug.Print("");

        // Get whether the width is a hole or a pin

        boolstatus = widthFeature.Inner;
        msgStr = "This width is for a hole: ";
        msgStr2 = boolstatus.ToString();
        Debug.Print(msgStr + msgStr2);
    }
}

public void GetPatternType(ref swDimXpertFeatureType_e featureType, ref string msgStr2)
{
    if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Plane))
    {
        msgStr2 = "Plane";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Cylinder))
    {
        msgStr2 = "Cylinder";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Cone))
    {
        msgStr2 = "Cone";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Extrude))
    {
        msgStr2 = "Extrude";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Fillet))
    {
        msgStr2 = "Fillet";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Chamfer))
    {
        msgStr2 = "Chamfer";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_CompoundHole))
    {
        msgStr2 = "CompoundHole";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_CompoundWidth))
    {
        msgStr2 = "CompoundWidth";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_CompoundNotch))
    {
        msgStr2 = "CompoundNotch";
    }
    else if ((featureType ==

```

```

swDimXpertFeatureType_e.swDimXpertFeature_CompoundClosedSlot3D))
{
    msgStr2 = "CompoundClosedSlot3D";
}
else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_IntersectPoint))
{
    msgStr2 = "IntersectPoint";
}
else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_IntersectLine))
{
    msgStr2 = "IntersectLine";
}
else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_IntersectCircle))
{
    msgStr2 = "IntersectCircle";
}
else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_IntersectPlane))
{
    msgStr2 = "IntersectPlane";
}
else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Pattern))
{
    msgStr2 = "Pattern";
}
else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Sphere))
{
    msgStr2 = "Sphere";
}
else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_BestfitPlane))
{
    msgStr2 = "Bestfit Plane";
}
else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Surface))
{
    msgStr2 = "Surface";
}

}

}

public SldWorks swApp;

}

```

Get and Set Size Dimension Example (VBA)

This example shows how to get and set DimXpert compound width features.

```
' -----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\dimxpert\bracket_ads_plusminus.sldprt.  
' 2. Select an edge.  
' 3. Open the Immediate window.  
' 4. Ensure that the latest SOLIDWORKS DimXpert type library is loaded:  
'     a. Select Tools > References.  
'     b. Click Browse.  
'     c. Find and select install_dir\swdimxpert.tlb.  
'  
' Postconditions:  
' 1. Inspect the Immediate Window.  
' 2. Observe the Width1 feature and the Width2 size dimension on the  
'     DimXpertManager tab.  
'  
' NOTE: Because this part is used elsewhere, do not save changes.  
' -----
```

Option Explicit

Dim dimXpertPart As dimXpertPart

Sub Main()

```
    Dim swapp As SldWorks.SldWorks  
    Set swapp = Application.SldWorks  
    Dim swModelDoc As SldWorks.ModelDoc2  
    Set swModelDoc = swapp.ActiveDoc  
    Dim annoType As Long  
    Dim swFeature As feature  
    Dim featureType As swDimXpertFeatureType_e  
    Dim features As Variant  
    Dim appliedFeatures As Variant  
    Dim appliedAnnotations As Variant  
    Dim appliedAnnotation As DimXpertAnnotation  
    Dim feature As DimXpertFeature  
    Dim appliedFeature As DimXpertFeature  
    Dim msgStr As String  
    Dim msgStr2 As String  
    Dim msgStr3 As String  
    Dim msgStr4 As String  
    Dim n As Long  
    Dim o As Long  
    Dim p As Long  
    Dim boolstatus As Boolean
```

If swModelDoc Is Nothing Then

Exit Sub

End If

```
    Dim dimXpertMgr As SldWorks.DimXpertManager  
    Set dimXpertMgr =
```

swapp.IActiveDoc2.Extension.DimXpertManager(swapp.IActiveDoc2.IGetActiveConfiguration().Name,
True)

```
    Debug.Print "Model: " & swapp.IActiveDoc2.GetPathName
```

```
    Dim dimXpertPartObj As dimXpertPart  
    Set dimXpertPartObj = dimXpertMgr.dimXpertPart  
    Set dimXpertPart = dimXpertPartObj
```

```
    Dim dimOption As DimXpertDimensionOption  
    Set dimOption = dimXpertPart.GetDimOption
```

```

' Specify the position of the annotation
Dim textPos(2) As Double
textPos(0) = 0.05
textPos(1) = -0.03
textPos(2) = -0.03
Dim posvar As Variant
posvar = textPos
dimOption.TextPosition = posvar

Dim dimarray(0) As Long
dimarray(0) = -1
Dim dimvar As Variant
dimvar = dimarray
dimOption.FeatureSelectorOptions = dimvar

' Insert the size dimension
dimXpertPart.InsertSizeDimension dimOption

Dim featCount As Long
featCount = dimXpertPart.GetFeatureCount

msgStr = "Total of "
msgStr2 = featCount
msgStr = msgStr + msgStr2 + " DimXpert features"
Debug.Print ""
Debug.Print msgStr

' Get IDimXpert features through IDimXpertPart
features = dimXpertPart.GetFeatures

msgStr = "  Features: "
Debug.Print ""
Debug.Print msgStr

For n = 0 To UBound(features)
    Set feature = features(n)
    Debug.Print "    " + "Feature name: " + (feature.Name)
    featureType = feature.Type
    Call GetPatternType(featureType, msgStr2)

    msgStr = "      Feature type "
    msgStr3 = " is suppressed on the DimXpertManager tab? "
    msgStr4 = feature.IsSuppressed()
    Debug.Print msgStr + msgStr2 + msgStr3 + msgStr4

    msgStr = "        " + "Model feature: "

    Set swFeature = feature.GetModelFeature()

    If Not (swFeature Is Nothing) Then
        msgStr2 = swFeature.GetTypeName2()
        Debug.Print msgStr + msgStr2
    End If

    msgStr = "          " + "Number of SOLIDWORKS face entities in this feature: "
    msgStr2 = feature.GetFaceCount
    Debug.Print msgStr + msgStr2

    msgStr = "          " + "Number of applied features: "
    msgStr2 = feature.GetAppliedFeatureCount()
    Debug.Print msgStr + msgStr2

    appliedFeatures = feature.GetAppliedFeatures()

    If Not (IsEmpty(appliedFeatures)) Then
        For o = 0 To UBound(appliedFeatures)
            Set appliedFeature = appliedFeatures(o)
            Debug.Print "            " + "Applied feature name: " + (appliedFeature.Name)
        Next
    End If

    msgStr = "          " + "Number of applied annotations: "
    msgStr2 = feature.GetAppliedAnnotationCount()
    Debug.Print msgStr + msgStr2

```

```

        appliedAnnotations = feature.GetAppliedAnnotations()
    If Not (IsEmpty(appliedAnnotations)) Then
        For p = 0 To UBound(appliedAnnotations)
            Set appliedAnnotation = appliedAnnotations(p)
            Debug.Print "        " + "Applied annotation name: " + (appliedAnnotation.Name)
        Next
    End If
    Debug.Print "        "
Next

' Get specific information about the Width1 feature
Dim widthFeature As IDimXpertCompoundWidthFeature
Set widthFeature = dimXpertPart.GetFeature("Width1")
msgStr = widthFeature.Name + " is a DimXpert compound width feature."
Debug.Print ""
Debug.Print msgStr
Debug.Print ""

' Get the nominal width coordinates
Dim width As Double
Dim x As Double
Dim y As Double
Dim z As Double
Dim i As Double
Dim j As Double
Dim k As Double

Debug.Print "Nominal width of Width1"
Debug.Print ""
boolstatus = widthFeature.GetNominalCompoundWidth(width, x, y, z, i, j, k)

msgStr = "Width is "
msgStr2 = width
Debug.Print msgStr + msgStr2

msgStr = "X-coordinate is "
msgStr2 = x
Debug.Print msgStr + msgStr2

msgStr = "Y-coordinate is "
msgStr2 = y
Debug.Print msgStr + msgStr2

msgStr = "Z-coordinate is "
msgStr2 = z
Debug.Print msgStr + msgStr2

msgStr = "I-component of pierce vector is "
msgStr2 = i
Debug.Print msgStr + msgStr2

msgStr = "J-component of pierce vector is "
msgStr2 = j
Debug.Print msgStr + msgStr2

msgStr = "K-component of pierce vector is "
msgStr2 = k
Debug.Print msgStr + msgStr2
Debug.Print ""

' Get whether the width is a hole or a pin
boolstatus = widthFeature.Inner
msgStr = "The width is for a hole and not a pin: "
msgStr2 = boolstatus
Debug.Print msgStr + msgStr2
End Sub

Public Sub GetPatternType(ByRef featureType, ByRef msgStr2)
    If (featureType = swDimXpertFeature_Plane) Then
        msgStr2 = "Plane"
    ElseIf (featureType = swDimXpertFeature_Cylinder) Then
        msgStr2 = "Cylinder"
    ElseIf (featureType = swDimXpertFeature_Cone) Then

```

```

        msgStr2 = "Cone"
    ElseIf (featureType = swDimXpertFeature_Extrude) Then
        msgStr2 = "Extrude"
    ElseIf (featureType = swDimXpertFeature_Fillet) Then
        msgStr2 = "Fillet"
    ElseIf (featureType = swDimXpertFeature_Chamfer) Then
        msgStr2 = "Chamfer"
    ElseIf (featureType = swDimXpertFeature_CompoundHole) Then
        msgStr2 = "CompoundHole"
    ElseIf (featureType = swDimXpertFeature_CompoundWidth) Then
        msgStr2 = "CompoundWidth"
    ElseIf (featureType = swDimXpertFeature_CompoundNotch) Then
        msgStr2 = "CompoundNotch"
    ElseIf (featureType = swDimXpertFeature_CompoundClosedSlot3D) Then
        msgStr2 = "CompoundClosedSlot3D"
    ElseIf (featureType = swDimXpertFeature_IntersectPoint) Then
        msgStr2 = "IntersectPoint"
    ElseIf (featureType = swDimXpertFeature_IntersectLine) Then
        msgStr2 = "IntersectLine"
    ElseIf (featureType = swDimXpertFeature_IntersectCircle) Then
        msgStr2 = "IntersectCircle"
    ElseIf (featureType = swDimXpertFeature_IntersectPlane) Then
        msgStr2 = "IntersectPlane"
    ElseIf (featureType = swDimXpertFeature_Pattern) Then
        msgStr2 = "Pattern"
    ElseIf (featureType = swDimXpertFeature_Sphere) Then
        msgStr2 = "Sphere"
    ElseIf (featureType = swDimXpertFeature_BestfitPlane) Then
        msgStr2 = "Bestfit Plane"
    ElseIf (featureType = swDimXpertFeature_Surface) Then
        msgStr2 = "Surface"
    End If

```

End Sub

Get and Set Size Dimension Example (VB.NET)

This example shows how to get and set DimXpert compound width features.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\dimxpert\bracket_ads_plusminus.sldprt.  
' 2. Select an edge.  
' 3. Open the Immediate window.  
' 4. Ensure that the latest SOLIDWORKS DimXpert interop assembly is referenced:  
'    a. Right-click the project in Project Explorer.  
'    b. Select Add Reference.  
'    c. Click the Browse tab.  
'    d. Find and select install_dir\api\redist\swdimxpert.dll.  
'  
' Postconditions:  
' 1. Inspect the Immediate Window.  
' 2. Observe the Width1 feature and the Width2 size dimension on the  
'    DimXpertManager tab.  
'  
' NOTE: Because this part is used elsewhere, do not save changes.  
'-----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports SolidWorks.Interop.swdimxpert  
Imports System  
Imports System.Diagnostics  
  
Partial Class SolidWorksMacro  
  
    Dim dimXpertPart As dimXpertPart  
  
    Sub Main()  
  
        Dim swModelDoc As ModelDoc2  
        swModelDoc = swapp.ActiveDoc  
        Dim swFeature As feature  
        Dim featureType As swDimXpertFeatureType_e  
        Dim features As Object  
        Dim appliedFeatures As Object  
        Dim appliedAnnotations As Object  
        Dim appliedAnnotation As DimXpertAnnotation  
        Dim feature As DimXpertFeature  
        Dim appliedFeature As DimXpertFeature  
        Dim msgStr As String  
        Dim msgStr2 As String  
        Dim msgStr3 As String  
        Dim msgStr4 As String  
        Dim n As Long  
        Dim o As Long  
        Dim p As Long  
        Dim boolstatus As Boolean  
  
        If swModelDoc Is Nothing Then  
            Exit Sub  
        End If  
  
        Dim dimXpertMgr As DimXpertManager  
        dimXpertMgr =  
swApp.IActiveDoc2.Extension.DimXpertManager(swApp.IActiveDoc2.IGetActiveConfiguration().Name,  
True)  
        Debug.Print("Model: " & swapp.IActiveDoc2.GetPathName)
```

```

Dim dimXpertPartObj As dimXpertPart
dimXpertPartObj = dimXpertMgr.dimXpertPart
dimXpertPart = dimXpertPartObj

Dim dimOption As DimXpertDimensionOption
dimOption = dimXpertPart.GetDimOption

' Specify the position of the dimension annotation
Dim textPos(2) As Double
textPos(0) = 0.05
textPos(1) = -0.03
textPos(2) = -0.03
Dim posvar As Object
posvar = textPos
dimOption.TextPosition = posvar

Dim dimarray(0) As Long
dimarray(0) = -1 ' default feature
Dim dimvar As Object
dimvar = dimarray
dimOption.FeatureSelectorOptions = dimvar

' Insert the size dimension
dimXpertPart.InsertSizeDimension(dimOption)

Dim featCount As Long
featCount = dimXpertPart.GetFeatureCount

msgStr = "Total of "
msgStr2 = featCount
msgStr = msgStr + msgStr2 + " DimXpert features"
Debug.Print("")
Debug.Print(msgStr)

' Get IDimXpert features through IDimXpertPart
features = dimXpertPart.GetFeatures
If Not IsNothing(features) Then
    msgStr = " Features: "
    Debug.Print("")
    Debug.Print(msgStr)

    For n = 0 To UBound(features)
        feature = features(n)
        Debug.Print(" " + "Feature name: " + (feature.Name))
        featureType = feature.Type
        Call GetPatternType(featureType, msgStr2)

        msgStr = " Feature type "
        msgStr3 = " is suppressed on the DimXpertManager tab? "
        msgStr4 = feature.IsSuppressed()
        Debug.Print(msgStr + msgStr2 + msgStr3 + msgStr4)

        msgStr = " " + "Model feature: "

        swFeature = feature.GetModelFeature()

        If Not (swFeature Is Nothing) Then
            msgStr2 = swFeature.GetTypeName2()
            Debug.Print(msgStr + msgStr2)
        End If

        msgStr = " " + "Number of SOLIDWORKS face entities in this feature: "
        msgStr2 = feature.GetFaceCount
        Debug.Print(msgStr + msgStr2)

        msgStr = " " + "Number of applied features: "
        msgStr2 = feature.GetAppliedFeatureCount()
        Debug.Print(msgStr + msgStr2)
    
```

```

        appliedFeatures = feature.GetAppliedFeatures()

        If Not IsNothing(appliedFeatures) Then
            For o = 0 To UBound(appliedFeatures)
                appliedFeature = appliedFeatures(o)
                Debug.Print("        " + "Applied feature name: " +
(appliedFeature.Name))
            Next
        End If

        msgStr = "        " + "Number of applied annotations: "
        msgStr2 = feature.GetAppliedAnnotationCount()
        Debug.Print(msgStr + msgStr2)

        appliedAnnotations = feature.GetAppliedAnnotations()

        If Not IsNothing(appliedAnnotations) Then
            For p = 0 To UBound(appliedAnnotations)
                appliedAnnotation = appliedAnnotations(p)
                Debug.Print("        " + "Applied annotation name: " +
(appliedAnnotation.Name))
            Next
        End If
        Debug.Print("        ")
    Next

    ' Get specific information about the Width1 feature

    Dim widthFeature As IDimXpertCompoundWidthFeature
    widthFeature = dimXpertPart.GetFeature("Width1")
    msgStr = widthFeature.Name + " is a DimXpert compound width feature."
    Debug.Print("")
    Debug.Print(msgStr)
    Debug.Print("")

    ' Get the nominal width coordinates

    Dim width As Double
    Dim x As Double
    Dim y As Double
    Dim z As Double
    Dim i As Double
    Dim j As Double
    Dim k As Double

    Debug.Print("Nominal width of Width1")
    Debug.Print("")
    boolStatus = widthFeature.GetNominalCompoundWidth(width, x, y, z, i, j, k)

    msgStr = "Width is "
    msgStr2 = width
    Debug.Print(msgStr + msgStr2)

    msgStr = "X-coordinate is "
    msgStr2 = x
    Debug.Print(msgStr + msgStr2)

    msgStr = "Y-coordinate is "
    msgStr2 = y
    Debug.Print(msgStr + msgStr2)

    msgStr = "Z-coordinate is "
    msgStr2 = z
    Debug.Print(msgStr + msgStr2)

    msgStr = "I-component of pierce vector is "
    msgStr2 = i
    Debug.Print(msgStr + msgStr2)

    msgStr = "J-component of pierce vector is "

```

```

msgStr2 = j
Debug.Print(msgStr + msgStr2)

msgStr = "K-component of pierce vector is "
msgStr2 = k
Debug.Print(msgStr + msgStr2)
Debug.Print("")

' Get whether the width is a hole or a pin

boolstatus = widthFeature.Inner
msgStr = "This width is for a hole: "
msgStr2 = boolstatus
Debug.Print(msgStr + msgStr2)
End If
End Sub

Public Sub GetPatternType(ByRef featureType As swDimXpertFeatureType_e, ByRef msgStr2 As String)

If (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Plane) Then
    msgStr2 = "Plane"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Cylinder) Then
    msgStr2 = "Cylinder"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Cone) Then
    msgStr2 = "Cone"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Extrude) Then
    msgStr2 = "Extrude"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Fillet) Then
    msgStr2 = "Fillet"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Chamfer) Then
    msgStr2 = "Chamfer"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundHole) Then
    msgStr2 = "CompoundHole"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundWidth) Then
    msgStr2 = "CompoundWidth"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundNotch) Then
    msgStr2 = "CompoundNotch"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundClosedSlot3D)
Then
    msgStr2 = "CompoundClosedSlot3D"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectPoint) Then
    msgStr2 = "IntersectPoint"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectLine) Then
    msgStr2 = "IntersectLine"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectCircle) Then
    msgStr2 = "IntersectCircle"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectPlane) Then
    msgStr2 = "IntersectPlane"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Pattern) Then
    msgStr2 = "Pattern"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Sphere) Then
    msgStr2 = "Sphere"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_BestfitPlane) Then
    msgStr2 = "Bestfit Plane"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Surface) Then
    msgStr2 = "Surface"
End If

End Sub

Public swApp As SldWorks

End Class

```

Get_DimXpert_Block_Tolerance_Example_VB (Topic)

SOLIDWORKS API Help

Get DimXpert Block Tolerance Example (VBA)

This example shows how to get the following block tolerance values:

- * ASME
- * ISO Code

```
' -----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\shaft.sldprt.  
' 2. Open an Immediate window.  
' 3. Ensure that the latest SOLIDWORKS DimXpert type library is loaded  
'    in Tools > References.  
' 4. Ensure that the Microsoft Scripting Runtime library is loaded  
'    in Tools > References.  
'  
' Postconditions:  
' 1. Inspect the Immediate window to see the ISO code for the part.  
' 2. Open public_documents\api\cover_with_geometric_tolerances.sldprt.  
' 3. Run this macro (F5).  
' 4. Inspect the Immediate Window to see the ASME block tolerance values  
'    for the part.  
' 5. Logs the output of this macro in c:\temp\dimXpertInfo.txt.  
' 6. Inspect the Immediate window.  
'  
' NOTE: Because the parts are used elsewhere, do not save changes.  
' -----
```

Option Explicit

Sub Main()

Dim swapp As SldWorks.SldWorks

Set swapp = Application.SldWorks

Dim swModelDoc As SldWorks.ModelDoc2

Set swModelDoc = swapp.ActiveDoc

If swModelDoc Is Nothing Then

Exit Sub

End If

Dim f As New FileSystemObject

Dim textStr As TextStream

Set textStr = f.CreateTextFile("C:\temp\dimXpertInfo.txt", True)

If textStr Is Nothing Then

Debug.Print "Error creating temp file."

Exit Sub

End If

Call log("-----", textStr)

```

    Call log("Starting DimXpert log...", textStr)
    Call retrieve_info_text(swapp, textStr)
    textStr.Close
End Sub

Private Sub log(text As String, textStr As TextStream)
    Debug.Print text
    textStr.WriteLine (text)
End Sub

Private Sub retrieve_info_text(swapp As SldWorks.SldWorks, textStr As TextStream)
    Dim dimXpertMgr As SldWorks.DimXpertManager

    Set dimXpertMgr =
swapp.IActiveDoc2.Extension.DimXpertManager(swapp.IActiveDoc2.IGetActiveConfiguration().Name,
True)

    Call log("Model: " & swapp.IActiveDoc2.GetPathName, textStr)
    Dim dimXpertPartObj As dimXpertPart
    Set dimXpertPartObj = dimXpertMgr.dimXpertPart

    Dim dimXpertPart As SwDimXpert.dimXpertPart
    Set dimXpertPart = dimXpertPartObj

    Dim vAnnotations As Variant

    vAnnotations = dimXpertPart.GetAnnotations()

    Call log("-----", textStr)
    Call log("Block Tolerances...", textStr)
    Call log("-----", textStr)

    Call listBlockTolerances_text(dimXpertPart, textStr)

End Sub

Private Sub listBlockTolerances_text(dimXpertPart As SwDimXpert.dimXpertPart, textStr As
TextStream)
    Dim blockTols As SwDimXpert.DimXpertBlockTolerances
    Dim boolstatus As Boolean
    Dim lin1 As Double, lin1prec As Long
    Dim lin2 As Double, lin2prec As Long
    Dim lin3 As Double, lin3prec As Long
    Dim ang As Double
    Dim isoCode As Long

    Set blockTols = dimXpertPart.GetBlockTolerances()
    If Not blockTols Is Nothing Then
        Select Case blockTols.Type

            Case swDimXpertBlockToleranceType_ASMEInch

```

```

lin3prec, ang)
    boolstatus = blockTols.GetToleranceValues(lin1, lin1prec, lin2, lin2prec, lin3,
lin3prec, ang)
    Call log("swDimXpertBlockToleranceType_ASMEInch", textStr)
    Call log( _
        "Linear1: " + Format(lin1prec) + " Places = " + Format(lin1, "##0.000000") +
"          " + _
        "Linear3: " + Format(lin3prec) + " Places = " + Format(lin3, "##0.000000") +
vbNewLine + _
        "Linear2: " + Format(lin2prec) + " Places = " + Format(lin2, "##0.000000") +
"          " + _
        "Angular = " + Format(ang * 57.2957795130823, "##0.000000"), textStr)

    Case swDimXpertBlockToleranceType_ISO2768
    Call log("swDimXpertBlockToleranceType_ISO2768", textStr)
        boolstatus = blockTols.GetISO2768PartType(isoCode)
        Select Case isoCode

            Case swDimXpertISO2768PartType_Fine
                Call log("General Tolerance: Fine", textStr)

            Case swDimXpertISO2768PartType_Medium
                Call log("General Tolerance: Medium", textStr)

            Case swDimXpertISO2768PartType_Coarse
                Call log("General Tolerance: Coarse", textStr)

            Case swDimXpertISO2768PartType_VeryCoarse
                Call log("General Tolerance: Very Coarse", textStr)

        End Select
    End Select
End If

End Sub

```

Get DimXpert Block Tolerance Example (VB.NET)

This example shows how to build and get attributes for the following DimXpert features:

- * Hole
- * Notch

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\cover_with_dimensions.sldprt.  
' 2. Open an Immediate window.  
' 3. Ensure that the SolidWorks.Interop.swdimxpert.dll interop  
'    is loaded (right-click the project in the Project Explorer,  
'    click Add Reference > .NET tab).  
' 4. Ensure that the Microsoft Scripting Runtime library is loaded  
'    (right-click the project in the Project Explorer and click  
'    Add Reference > COM tab).  
'  
' Postconditions:  
' 1. Inspect the Immediate window to see the ISO code for the part.  
' 2. Open public_documents\samples\tutorial\api\cover_with_geometric_tolerances.sldprt.  
' 3. Run this macro (F5).  
' 4. Inspect the Immediate Window to see the ASME block tolerance values  
'    for the part.  
' 5. Logs the output of this macro in c:\temp\dimXpertInfo.txt.  
' 6. Inspect the Immediate window.  
'  
' NOTE: Because the parts are used elsewhere, do not save changes.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swdimxpert  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Diagnostics  
Imports Scripting  
Partial Class SolidWorksMacro  
    Public Sub main()  
  
        Dim swModelDoc As ModelDoc2  
        swModelDoc = swapp.ActiveDoc  
        If swModelDoc Is Nothing Then  
            Exit Sub  
        End If  
        Dim f As New FileSystemObject  
        Dim textStr As TextStream
```

```

textStr = f.CreateTextFile("C:\temp\dimXpertInfo.txt", True)
If textStr Is Nothing Then
    Debug.Print("Error creating temp file.")
    Exit Sub
End If
Call log("-----", textStr)
Call log("Starting DimXpert log...", textStr)
Call retrieve_info_text(swapp, textStr)
textStr.Close()
End Sub

Private Sub log(ByVal text As String, ByVal textStr As TextStream)
    Debug.Print(text)
    textStr.WriteLine(text)
End Sub

Private Sub retrieve_info_text(ByVal swapp As SldWorks, ByVal textStr As TextStream)
    Dim dimXpertMgr As DimXpertManager
    dimXpertMgr =
swapp.IActiveDoc2.Extension.DimXpertManager(swapp.IActiveDoc2.IGetActiveConfiguration().Name,
True)

    Call log("Model: " & swapp.IActiveDoc2.GetPathName, textStr)
    Dim dimXpertPartObj As DimXpertPart
    dimXpertPartObj = dimXpertMgr.DimXpertPart
    Dim dimXpertPart As DimXpertPart
    dimXpertPart = dimXpertPartObj
    Dim vAnnotations As Object
    vAnnotations = dimXpertPart.GetAnnotations()
    Call log("-----", textStr)
    Call log("Block Tolerances...", textStr)
    Call log("-----", textStr)
    Call listBlockTolerances_text(dimXpertPart, textStr)
End Sub

Private Sub listBlockTolerances_text(ByVal dimXpertPart As DimXpertPart, ByVal textStr As
TextStream)
    Dim blockTols As DimXpertBlockTolerances
    Dim boolstatus As Boolean
    Dim lin1 As Double, lin1prec As Long
    Dim lin2 As Double, lin2prec As Long
    Dim lin3 As Double, lin3prec As Long
    Dim ang As Double
    Dim isoCode As Long
    blockTols = dimXpertPart.GetBlockTolerances()
    If Not blockTols Is Nothing Then
        Select Case blockTols.Type
            Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ASMEInch
                boolstatus = blockTols.GetToleranceValues(lin1, lin1prec, lin2, lin2prec,
lin3, lin3prec, ang)
                Call log("swDimXpertBlockToleranceType_ASMEInch", textStr)
                Call log( _

```

```

        "Linear1: " + Format(lin1prec) + " Places = " + Format(lin1,
        "##0.000000") + " " + _
        "Linear3: " + Format(lin3prec) + " Places = " + Format(lin3,
        "##0.000000") + vbNewLine + _
        "Linear2: " + Format(lin2prec) + " Places = " + Format(lin2,
        "##0.000000") + " " + _
        "Angular = " + Format(ang * 57.2957795130823, "##0.000000"), textStr)
    Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_IS02768
        Call log("swDimXpertBlockToleranceType_IS02768", textStr)
        boolstatus = blockTols.GetIS02768PartType(isoCode)
        Select Case isoCode
            Case swDimXpertIS02768PartType_e.swDimXpertIS02768PartType_Fine
                Call log("General Tolerance: Fine", textStr)
            Case swDimXpertIS02768PartType_e.swDimXpertIS02768PartType_Medium
                Call log("General Tolerance: Medium", textStr)
            Case swDimXpertIS02768PartType_e.swDimXpertIS02768PartType_Coarse
                Call log("General Tolerance: Coarse", textStr)
            Case swDimXpertIS02768PartType_e.swDimXpertIS02768PartType_VeryCoarse
                Call log("General Tolerance: Very Coarse", textStr)
        End Select
    End Select
End If
End Sub

Public swApp As SldWorks
End Class

```

Get DimXpert Chamfer Feature Example (VBA)

This example shows how to build and get attributes for the DimXpert chamfer feature.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\apil\plate_tolstatus.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars.  
' 3. Click the Auto Dimension Scheme icon in the DimXpert toolbar.  
' 4. Ensure that the chamfer feature filter is selected.  
' 5. Click the green check mark to accept all settings.  
' 6. Observe this DimXpert features on the DimXpertManager tab  
'   of the Management Panel: Chamfer1.  
' 7. Open an Immediate window.  
' 8. Ensure that the latest SOLIDWORKS DimXpert type library is loaded  
'   in Tools > References.  
' 9. Step through this macro (F8).  
'  
' Postconditions: Compare the output in the Immediate window with  
' the features displayed on the DimXpertManager tab of the Management Panel.  
'  
' NOTE: Because this part is used elsewhere, do not save changes.  
'-----  
  
Option Explicit  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swConfig As SldWorks.Configuration  
Dim swFeature As SldWorks.feature  
Dim swAnn As SldWorks.feature  
Dim swSchema As SldWorks.DimXpertManager  
Dim swDXPart As DimXpertPart  
Dim featureType As swDimXpertFeatureType_e  
Dim chamferType As swDimXpertChamferType_e  
Dim chamferAngleType As swDimXpertChamferAngleType_e  
Dim features As Variant  
Dim appliedFeatures As Variant  
Dim appliedAnnotations As Variant  
Dim appliedAnnotation As DimXpertAnnotation  
Dim feature As DimXpertFeature  
Dim appliedFeature As DimXpertFeature  
Dim msgStr As String  
Dim msgStr2 As String  
Dim msgStr3 As String  
Dim msgStr4 As String  
Dim n As Long  
Dim o As Long  
Dim p As Long
```

```

Dim boolstatus As Boolean
Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager

    ' Get the default DimXpert schema using IModelDocExtension.DimXpertManager()

    Set swSchema = swModelDocExt.DimXpertManager("Default", True)

    ' Get IDimXpertPart from the IDimXpertManager
    Set swDXPart = swSchema.DimXpertPart

    Dim featCount As Long
    featCount = swDXPart.GetFeatureCount
    msgStr = "Total of "
    msgStr2 = featCount
    msgStr = msgStr + msgStr2 + " features in " + (swSchema.SchemaName)
    Debug.Print ""
    Debug.Print msgStr

    ' Get IDimXpert features through IDimXpertPart

    features = swDXPart.GetFeatures

    msgStr = (swSchema.SchemaName) + " has the following features: "
    Debug.Print ""
    Debug.Print msgStr
    For n = 0 To UBound(features)
        Set feature = features(n)
        Debug.Print " " + "Feature name: " + (feature.Name)

        featureType = feature.Type
        Call GetPatternType(featureType, msgStr2)

        msgStr = " " + "Feature type "
        msgStr3 = " is suppressed on the DimXpertManager tab? "
        msgStr4 = feature.IsSuppressed()
        Debug.Print msgStr + msgStr2 + msgStr3 + msgStr4

        msgStr = " " + "Model feature: "
        Set swFeature = feature.GetModelFeature()
        If Not (swFeature Is Nothing) Then
            msgStr2 = swFeature.GetTypeNames2()
            Debug.Print msgStr + msgStr2
        End If

        msgStr = " " + "Number of SOLIDWORKS face entities in this feature: "
        msgStr2 = feature.GetFaceCount
        Debug.Print msgStr + msgStr2

        msgStr = " " + "Number of applied features: "

```

```

msgStr2 = feature.GetAppliedFeatureCount()
Debug.Print msgStr + msgStr2

appliedFeatures = feature.GetAppliedFeatures()
If Not (IsEmpty(appliedFeatures)) Then
    For o = 0 To UBound(appliedFeatures)
        Set appliedFeature = appliedFeatures(o)
        Debug.Print "    " + "Applied feature name: " + (appliedFeature.Name)
    Next
End If

msgStr = "    " + "Number of applied annotations: "
msgStr2 = feature.GetAppliedAnnotationCount()
Debug.Print msgStr + msgStr2

appliedAnnotations = feature.GetAppliedAnnotations()
If Not (IsEmpty(appliedAnnotations)) Then
    For p = 0 To UBound(appliedAnnotations)
        Set appliedAnnotation = appliedAnnotations(p)
        Debug.Print "    " + "Applied annotation name: " + (appliedAnnotation.Name)
    Next
End If

Debug.Print "    "
Next

' If you know the name of a DimXpert feature, you can get it directly using IDimXpertPart.GetFeature("name"),
' which can return a general IDimXpertFeature or a more specific interface on the feature

' Get IDimXpertChamferFeature for the Chamfer1 feature

Dim chamferFeature As IDimXpertChamferFeature
Set chamferFeature = swDXPart.GetFeature("Chamfer1")
msgStr = chamferFeature.Name + " is a DimXpert feature"
Debug.Print ""
Debug.Print msgStr
Debug.Print ""

' Get the chamfer type
chamferType = chamferFeature.chamferType
msgStr = "    Chamfer type is "
Call GetChamferType(chamferType, msgStr2)
Debug.Print msgStr + msgStr2

' Get the chamfer angle
Dim angle As Double
angle = chamferFeature.angle
msgStr = "    Chamfer angle is "
msgStr2 = angle
Debug.Print msgStr + msgStr2

' Get the angle type
chamferAngleType = chamferFeature.angleType
msgStr = "    Chamfer angle type is "

```

```
Call GetChamferAngleType(chamferAngleType, msgStr2)
Debug.Print msgStr + msgStr2
```

```
'Get the chamfer distance properties
Dim chamferDistance1 As Double
Dim chamferDistance2 As Double
chamferDistance1 = chamferFeature.Distance1
msgStr = "    Chamfer distance 1 is "
msgStr2 = chamferDistance1
Debug.Print msgStr + msgStr2
chamferDistance2 = chamferFeature.Distance2
msgStr = "    Chamfer distance 2 is "
msgStr2 = chamferDistance2
Debug.Print msgStr + msgStr2
```

End Sub

```
Public Sub GetPatternType(ByRef featureType, ByRef msgStr2)
    If (featureType = swDimXpertFeature_Plane) Then
        msgStr2 = "Plane"
    ElseIf (featureType = swDimXpertFeature_Cylinder) Then
        msgStr2 = "Cylinder"
    ElseIf (featureType = swDimXpertFeature_Cone) Then
        msgStr2 = "Cone"
    ElseIf (featureType = swDimXpertFeature_Extrude) Then
        msgStr2 = "Extrude"
    ElseIf (featureType = swDimXpertFeature_Fillet) Then
        msgStr2 = "Fillet"
    ElseIf (featureType = swDimXpertFeature_Chamfer) Then
        msgStr2 = "Chamfer"
    ElseIf (featureType = swDimXpertFeature_CompoundHole) Then
        msgStr2 = "CompoundHole"
    ElseIf (featureType = swDimXpertFeature_CompoundWidth) Then
        msgStr2 = "CompoundWidth"
    ElseIf (featureType = swDimXpertFeature_CompoundNotch) Then
        msgStr2 = "CompoundNotch"
    ElseIf (featureType = swDimXpertFeature_CompoundClosedSlot3D) Then
        msgStr2 = "CompoundClosedSlot3D"
    ElseIf (featureType = swDimXpertFeature_IntersectPoint) Then
        msgStr2 = "IntersectPoint"
    ElseIf (featureType = swDimXpertFeature_IntersectLine) Then
        msgStr2 = "IntersectLine"
    ElseIf (featureType = swDimXpertFeature_IntersectCircle) Then
        msgStr2 = "IntersectCircle"
    ElseIf (featureType = swDimXpertFeature_IntersectPlane) Then
        msgStr2 = "IntersectPlane"
    ElseIf (featureType = swDimXpertFeature_Pattern) Then
        msgStr2 = "Pattern"
    ElseIf (featureType = swDimXpertFeature_Sphere) Then
        msgStr2 = "Sphere"
    ElseIf (featureType = swDimXpertFeature_BestfitPlane) Then
        msgStr2 = "Bestfit plane"
    ElseIf (featureType = swDimXpertFeature_Surface) Then
        msgStr2 = "Surface"
```

End If

End Sub

Public Sub GetChamferType(ByRef chamferType, ByRef msgStr2)

If (chamferType = swDimXpertChamferType_DistanceAngle) Then

msgStr2 = "Distance-Angle"

ElseIf (chamferType = swDimXpertChamferType_DistanceDistance) Then

msgStr2 = "Distance-Distance"

ElseIf (chamferType = swDimXpertChamferType_Vertex) Then

msgStr2 = "Vertex"

End If

End Sub

Public Sub GetChamferAngleType(ByRef angleType, ByRef msgStr2)

If (angleType = swDimXpertChamferAngleType_Concave) Then

msgStr2 = "Concave"

ElseIf (angleType = swDimXpertChamferAngleType_Convex) Then

msgStr2 = "Convex"

End If

End Sub

Get DimXpert Chamfer Feature Example (VB.NET)

This example shows how to build a part and get attributes for the DimXpert chamfer feature.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\plate_tolstatus.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars.  
' 3. Click the Auto Dimension Scheme icon in the DimXpert toolbar.  
' 4. Ensure that the Chamfer Feature Filter is selected.  
' 5. Click the green check mark to accept the settings.  
' 6. Observe this DimXpert feature on the DimXpertManager tab  
'   of the Management Panel: Chamfer1.  
' 7. Open an Immediate window.  
' 8. Ensure that the SolidWorks.Interop.swdimxpert.dll interop  
'   is loaded (right-click project, click Add Reference > .NET tab).  
' 9. Step through this macro (press F8).  
'  
' Postconditions: Compare the output in the Immediate Window  
' with the features displayed on the DimXpertManager tab of the Management Panel.  
'  
' NOTE: Because this part is used elsewhere, do not save changes.  
'-----
```

Imports SolidWorks.Interop.sldworks

Imports SolidWorks.Interop.swdimxpert

Imports SolidWorks.Interop.swconst

Imports System

Imports System.Diagnostics

Partial Class SolidWorksMacro

Dim swModel As IModelDoc2

Dim swModelDocExt As IModelDocExtension

Dim swSelMgr As ISelectionMgr

Dim swConfig As IConfiguration

Dim swFeature As IFeature

Dim swAnn As IFeature

Dim swSchema As IDimXpertManager

Dim swDXPart As IDimXpertPart

Dim featureType As swDimXpertFeatureType_e

Dim chamferType As swDimXpertChamferType_e

Dim chamferAngleType As swDimXpertChamferAngleType_e

Dim features As Object

Dim appliedFeatures As Object

Dim appliedAnnotations As Object

Dim appliedAnnotation As IDimXpertAnnotation

Dim feature As IDimXpertFeature

Dim appliedFeature As IDimXpertFeature

Dim msgStr As String

Dim msgStr2 As String

Dim msgStr3 As String

Dim msgStr4 As String

```

Dim n As Long
Dim o As Long
Dim p As Long
Dim boolstatus As Boolean
Sub main()
    swModel = swApp.ActiveDoc
    swModelDocExt = swModel.Extension
    swSelMgr = swModel.SelectionManager
    ' Get the default DimXpert schema using IModelDocExtension.DimXpertManager()
    swSchema = swModelDocExt.DimXpertManager("Default", True)
    ' Get IDimXpertPart from the IDimXpertManager
    swDXPart = swSchema.DimXpertPart
    Dim featCount As Long
    featCount = swDXPart.GetFeatureCount
    msgStr = "Total of "
    msgStr2 = featCount
    msgStr = msgStr + msgStr2 + " features in " + (swSchema.SchemaName)
    Debug.Print("")
    Debug.Print(msgStr)
    ' Get IDimXpert features through IDimXpertPart
    features = swDXPart.GetFeatures
    msgStr = (swSchema.SchemaName) + " has the following features: "
    Debug.Print("")
    Debug.Print(msgStr)
    For n = 0 To UBound(features)
        feature = features(n)
        Debug.Print(" " + "Feature name: " + (feature.Name))
        featureType = feature.Type
        Call GetPatternType(featureType, msgStr2)
        msgStr = "    Feature type "
        msgStr3 = " is suppressed on the DimXpertManager tab? "
        msgStr4 = feature.IsSuppressed()
        Debug.Print(msgStr + msgStr2 + msgStr3 + msgStr4)
        msgStr = "    " + "Model feature: "
        swFeature = feature.GetModelFeature()
        If Not (swFeature Is Nothing) Then
            msgStr2 = swFeature.GetTypeNames2()
            Debug.Print(msgStr + msgStr2)
        End If
        msgStr = "    " + "Number of SOLIDWORKS face entities in this feature: "
        msgStr2 = feature.GetFaceCount
        Debug.Print(msgStr + msgStr2)
        msgStr = "    " + "Number of applied features: "
        msgStr2 = feature.GetAppliedFeatureCount()
        Debug.Print(msgStr + msgStr2)
        appliedFeatures = feature.GetAppliedFeatures()
        If Not (IsNothing(appliedFeatures)) Then
            For o = 0 To UBound(appliedFeatures)
                appliedFeature = appliedFeatures(o)
                Debug.Print("        " + "Applied feature name: " + (appliedFeature.Name))
            Next
        End If
        msgStr = "    " + "Number of applied annotations: "
        msgStr2 = feature.GetAppliedAnnotationCount()

```

```

Debug.Print(msgStr + msgStr2)
appliedAnnotations = feature.GetAppliedAnnotations()
If Not (IsNothing(appliedAnnotations)) Then
    For p = 0 To UBound(appliedAnnotations)
        appliedAnnotation = appliedAnnotations(p)
        Debug.Print("    " + "Applied annotation name: " + (appliedAnnotation.Name))
    Next
End If
Debug.Print(" ")
Next

' If you know the name of a DimXpert feature, you can get it directly using IDimXpertPart.GetFeature("name"),
' which can return a general IDimXpertFeature or a more specific interface on the feature
' Get IDimXpertChamferFeature for the Chamfer1 feature
Dim chamferFeature As IDimXpertChamferFeature
chamferFeature = swDXPart.GetFeature("Chamfer1")
msgStr = chamferFeature.Name + " is a DimXpert feature"
Debug.Print("")
Debug.Print(msgStr)
Debug.Print("")
' Get the chamfer type
chamferType = chamferFeature.chamferType
msgStr = "    Chamfer type is "
Call GetChamferType(chamferType, msgStr2)
Debug.Print(msgStr + msgStr2)
' Get the chamfer angle
Dim angle As Double
angle = chamferFeature.angle
msgStr = "    Chamfer angle is "
msgStr2 = angle
Debug.Print(msgStr + msgStr2)
' Get the angle type
chamferAngleType = chamferFeature.angleType
msgStr = "    Chamfer angle type is "
Call GetChamferAngleType(chamferAngleType, msgStr2)
Debug.Print(msgStr + msgStr2)
'Get the chamfer distance properties
Dim chamferDistance1 As Double
Dim chamferDistance2 As Double
chamferDistance1 = chamferFeature.Distance1
msgStr = "    Chamfer distance 1 is "
msgStr2 = chamferDistance1
Debug.Print(msgStr + msgStr2)
chamferDistance2 = chamferFeature.Distance2
msgStr = "    Chamfer distance 2 is "
msgStr2 = chamferDistance2
Debug.Print(msgStr + msgStr2)
End Sub

Public Sub GetPatternType(ByRef featureType As swDimXpertFeatureType_e, ByRef msgStr2 As String)
    If (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_Plane) Then
        msgStr2 = "Plane"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_Cylinder) Then
        msgStr2 = "Cylinder"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_Cone) Then
        msgStr2 = "Cone"
    End If
End Sub

```

```

ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Extrude) Then
    msgStr2 = "Boss"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Fillet) Then
    msgStr2 = "Fillet"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Chamfer) Then
    msgStr2 = "Chamfer"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundHole) Then
    msgStr2 = "CompoundHole"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundWidth) Then
    msgStr2 = "CompoundWidth"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundNotch) Then
    msgStr2 = "CompoundNotch"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundClosedSlot3D) Then
    msgStr2 = "CompoundClosedSlot3D"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectPoint) Then
    msgStr2 = "IntersectPoint"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectLine) Then
    msgStr2 = "IntersectLine"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectCircle) Then
    msgStr2 = "IntersectCircle"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectPlane) Then
    msgStr2 = "IntersectPlane"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Pattern) Then
    msgStr2 = "Pattern"
        ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Sphere) Then
            msgStr2 = "Sphere"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_BestfitPlane) Then
    msgStr2 = "Bestfit plane"
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Surface) Then
    msgStr2 = "Surface"
End If
End Sub
Public Sub GetChamferType(ByRef chamferType As swDimXpertChamferType_e, ByRef msgStr2 As String)
    If (chamferType = swDimXpertChamferType_e.swDimXpertChamferType_DistanceAngle) Then
        msgStr2 = "Distance-Angle"
    ElseIf (chamferType = swDimXpertChamferType_e.swDimXpertChamferType_DistanceDistance) Then
        msgStr2 = "Distance-Distance"
    ElseIf (chamferType = swDimXpertChamferType_e.swDimXpertChamferType_Vertex) Then
        msgStr2 = "Vertex"
    End If
End Sub
Public Sub GetChamferAngleType(ByRef angleType As swDimXpertChamferAngleType_e, ByRef msgStr2 As String)
    If (angleType = swDimXpertChamferAngleType_e.swDimXpertChamferAngleType_Concave) Then
        msgStr2 = "Concave"
    ElseIf (angleType = swDimXpertChamferAngleType_e.swDimXpertChamferAngleType_Convex) Then
        msgStr2 = "Convex"
    End If
End Sub
Public swApp As SldWorks
End Class

```

Get_DimXpert_Cone_Feature_Example_VB (Topic)

SOLIDWORKS API Help

Get DimXpert Cone Feature Example (VBA)

This example shows how to build a part and get attributes for the DimXpert Cone feature.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\apilshaft.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars.  
' 3. Click the Auto Dimension Scheme icon in the DimXpert toolbar.  
' 4. Ensure that the Cone Feature Filter is selected.  
' 5. Click the green check mark to accept the settings.  
' 6. Observe this DimXpert feature on the DimXpertManager tab  
'   of the Management Panel: Cone1.  
' 7. Open an Immediate window.  
' 8. Ensure that the latest SOLIDWORKS DimXpert type library is loaded  
'   in Tools > References.  
' 9. Step through this macro (press F8).  
'  
' Postconditions: Compare the output in the Immediate Window with  
' the features displayed on the DimXpertManager tab of the Management Panel.  
'  
' NOTE: Because this part is used elsewhere, do not save changes.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swConfig As SldWorks.Configuration  
Dim swFeature As SldWorks.feature  
Dim swAnn As SldWorks.feature  
Dim swSchema As SldWorks.DimXpertManager  
Dim swDXPart As DimXpertPart  
Dim featureType As swDimXpertFeatureType_e  
Dim features As Variant  
Dim appliedFeatures As Variant  
Dim appliedAnnotations As Variant  
Dim appliedAnnotation As DimXpertAnnotation  
Dim feature As DimXpertFeature  
Dim appliedFeature As DimXpertFeature  
Dim msgStr As String  
Dim msgStr2 As String  
Dim msgStr3 As String  
Dim msgStr4 As String  
Dim n As Long  
Dim o As Long  
Dim p As Long  
Dim boolstatus As Boolean  
Sub main()  
    Set swApp = Application.SldWorks
```

```

Set swModel = swApp.ActiveDoc

Set swModelDocExt = swModel.Extension
Set swSelMgr = swModel.SelectionManager

' Get the default DimXpert schema using IModelDocExtension.DimXpertManager()

Set swSchema = swModelDocExt.DimXpertManager("Default", True)

' Get IDimXpertPart from the IDimXpertManager
Set swDXPart = swSchema.DimXpertPart

Dim featCount As Long
featCount = swDXPart.GetFeatureCount
msgStr = "Total of "
msgStr2 = featCount
msgStr = msgStr + msgStr2 + " features in " + (swSchema.SchemaName)
Debug.Print ""
Debug.Print msgStr

' Get IDimXpert features through IDimXpertPart

features = swDXPart.GetFeatures

msgStr = (swSchema.SchemaName) + " has the following features: "
Debug.Print ""
Debug.Print msgStr
For n = 0 To UBound(features)
    Set feature = features(n)
    Debug.Print " " + "Feature name: " + (feature.Name)

    featureType = feature.Type
    Call GetPatternType(featureType, msgStr2)

    msgStr = "    Feature type "
    msgStr3 = " is suppressed on the DimXpertManager tab? "
    msgStr4 = feature.IsSuppressed()
    Debug.Print msgStr + msgStr2 + msgStr3 + msgStr4

    msgStr = "    " + "Model feature: "
    Set swFeature = feature.GetModelFeature()
    If Not (swFeature Is Nothing) Then
        msgStr2 = swFeature.GetTypeNames2()
        Debug.Print msgStr + msgStr2
    End If

    msgStr = "    " + "Number of SOLIDWORKS face entities in this feature: "
    msgStr2 = feature.GetFaceCount
    Debug.Print msgStr + msgStr2

    msgStr = "    " + "Number of applied features: "
    msgStr2 = feature.GetAppliedFeatureCount()
    Debug.Print msgStr + msgStr2

```

```

appliedFeatures = feature.GetAppliedFeatures()
If Not (IsEmpty(appliedFeatures)) Then
    For o = 0 To UBound(appliedFeatures)
        Set appliedFeature = appliedFeatures(o)
        Debug.Print "    " + "Applied feature name: " + (appliedFeature.Name)
    Next
End If

```

```

msgStr = "    " + "Number of applied annotations: "
msgStr2 = feature.GetAppliedAnnotationCount()
Debug.Print msgStr + msgStr2

```

```

appliedAnnotations = feature.GetAppliedAnnotations()
If Not (IsEmpty(appliedAnnotations)) Then
    For p = 0 To UBound(appliedAnnotations)
        Set appliedAnnotation = appliedAnnotations(p)
        Debug.Print "    " + "Applied annotation name: " + (appliedAnnotation.Name)
    Next
End If

```

```

Debug.Print "    "
Next

```

' If you know the name of a DimXpert feature, you can get it directly using IDimXpertPart.GetFeature("name"),
' which can return a general IDimXpertFeature or a more specific interface on the feature

' Get IDimXpertConeFeature for the Cone1 feature

```

Dim coneFeature As IDimXpertConeFeature
Set coneFeature = swDXPart.GetFeature("Cone1")
msgStr = coneFeature.Name + " is a DimXpert feature"
Debug.Print ""
Debug.Print msgStr
Debug.Print ""

```

' Get the nominal top plane coordinates for Cone1

```

Dim x As Double
Dim y As Double
Dim z As Double
Dim i As Double
Dim j As Double
Dim k As Double
Dim angle As Double

```

```

Debug.Print "Nominal top plane of Cone1"
Debug.Print ""
boolstatus = coneFeature.GetNominalTopPlane(x, y, z, i, j, k)
msgStr = "X-coordinate is "
msgStr2 = x
Debug.Print msgStr + msgStr2
msgStr = "Y-coordinate is "
msgStr2 = y
Debug.Print msgStr + msgStr2

```

```

msgStr = "Z-coordinate is "
msgStr2 = z
Debug.Print msgStr + msgStr2
msgStr = "I-component of unit vector is "
msgStr2 = i
Debug.Print msgStr + msgStr2
msgStr = "J-component of unit vector is "
msgStr2 = j
Debug.Print msgStr + msgStr2
msgStr = "K-component of unit vector is "
msgStr2 = k
Debug.Print msgStr + msgStr2
Debug.Print ""

```

```

' Get the nominal bottom plane coordinates for Cone1

```

```

Debug.Print "Nominal bottom plane of Cone1"
Debug.Print ""
boolstatus = coneFeature.GetNominalBottomPlane(x, y, z, i, j, k)
msgStr = "X-coordinate is "
msgStr2 = x
Debug.Print msgStr + msgStr2
msgStr = "Y-coordinate is "
msgStr2 = y
Debug.Print msgStr + msgStr2
msgStr = "Z-coordinate is "
msgStr2 = z
Debug.Print msgStr + msgStr2
msgStr = "I-component of unit vector is "
msgStr2 = i
Debug.Print msgStr + msgStr2
msgStr = "J-component of unit vector is "
msgStr2 = j
Debug.Print msgStr + msgStr2
msgStr = "K-component of unit vector is "
msgStr2 = k
Debug.Print msgStr + msgStr2
Debug.Print ""

```

```

' Get the nominal cone angle and coordinates for Cone1

```

```

Debug.Print "Nominal cone of Cone1"
Debug.Print ""
boolstatus = coneFeature.GetNominalCone(angle, x, y, z, i, j, k)
msgStr = "Angle is "
msgStr2 = (angle / 3.14159265358979) * 180
msgStr3 = " degrees"
Debug.Print msgStr + msgStr2 + msgStr3
msgStr = "X-coordinate is "
msgStr2 = x
Debug.Print msgStr + msgStr2
msgStr = "Y-coordinate is "
msgStr2 = y
Debug.Print msgStr + msgStr2

```

```

msgStr = "Z-coordinate is "
msgStr2 = z
Debug.Print msgStr + msgStr2
msgStr = "I-component of unit vector is "
msgStr2 = i
Debug.Print msgStr + msgStr2
msgStr = "J-component of unit vector is "
msgStr2 = j
Debug.Print msgStr + msgStr2
msgStr = "K-component of unit vector is "
msgStr2 = k
Debug.Print msgStr + msgStr2
Debug.Print ""

```

' Is the cone feature for a hole or a pin?

```

boolstatus = coneFeature.Inner
msgStr = "The cone is a hole and not a pin: "
msgStr2 = boolstatus
Debug.Print msgStr + msgStr2

```

End Sub

```

Public Sub GetPatternType(ByRef featureType, ByRef msgStr2)
    If (featureType = swDimXpertFeature_Plane) Then
        msgStr2 = "Plane"
    ElseIf (featureType = swDimXpertFeature_Cylinder) Then
        msgStr2 = "Cylinder"
    ElseIf (featureType = swDimXpertFeature_Cone) Then
        msgStr2 = "Cone"
    ElseIf (featureType = swDimXpertFeature_Extrude) Then
        msgStr2 = "Extrude"
    ElseIf (featureType = swDimXpertFeature_Fillet) Then
        msgStr2 = "Fillet"
    ElseIf (featureType = swDimXpertFeature_Chamfer) Then
        msgStr2 = "Chamfer"
    ElseIf (featureType = swDimXpertFeature_CompoundHole) Then
        msgStr2 = "CompoundHole"
    ElseIf (featureType = swDimXpertFeature_CompoundWidth) Then
        msgStr2 = "CompoundWidth"
    ElseIf (featureType = swDimXpertFeature_CompoundNotch) Then
        msgStr2 = "CompoundNotch"
    ElseIf (featureType = swDimXpertFeature_CompoundClosedSlot3D) Then
        msgStr2 = "CompoundClosedSlot3D"
    ElseIf (featureType = swDimXpertFeature_IntersectPoint) Then
        msgStr2 = "IntersectPoint"
    ElseIf (featureType = swDimXpertFeature_IntersectLine) Then
        msgStr2 = "IntersectLine"
    ElseIf (featureType = swDimXpertFeature_IntersectCircle) Then
        msgStr2 = "IntersectCircle"
    ElseIf (featureType = swDimXpertFeature_IntersectPlane) Then
        msgStr2 = "IntersectPlane"
    ElseIf (featureType = swDimXpertFeature_Pattern) Then
        msgStr2 = "Pattern"
    ElseIf (featureType = swDimXpertFeature_Sphere) Then

```

```
msgStr2 = "Sphere"
```

```
Elseif (featureType = swDimXpertFeature_BestfitPlane) Then
```

```
msgStr2 = "Bestfit plane"
```

```
Elseif (featureType = swDimXpertFeature_Surface) Then
```

```
msgStr2 = "Surface"
```

```
End If
```

```
End Sub
```

Get DimXpert Cone Feature Example (VB.NET)

This example shows how to build a part and get attributes for the DimXpert cone feature.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\shaft.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars.  
' 3. Click the Auto Dimension Scheme icon in the DimXpert toolbar.  
' 4. Ensure that the Cone Feature Filter is selected.  
' 5. Click the green check mark to accept the settings.  
' 6. Observe this DimXpert feature on the DimXpertManager tab  
'   of the Management Panel: Cone1.  
' 7. Open an Immediate window.  
' 8. Ensure that the SolidWorks.Interop.swdimxpert.dll interop  
'   is loaded (right-click project in the Project Explorer and  
'   click Add Reference > .NET tab).  
' 9. Step through this macro (press F8).  
'  
' Postconditions: Compare the output in the Immediate Window with  
' the features displayed on the DimXpertManager tab of the Management Panel.  
'  
' NOTE: Because this part is used elsewhere, do not save changes.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swdimxpert  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Diagnostics  
Partial Class SolidWorksMacro  
    Dim swModel As IModelDoc2  
    Dim swModelDocExt As IModelDocExtension  
    Dim swSelMgr As ISelectionMgr  
    Dim swConfig As IConfiguration  
    Dim swFeature As IFeature  
    Dim swAnn As IFeature  
    Dim swSchema As IDimXpertManager  
    Dim swDXPart As IDimXpertPart  
    Dim featureType As swDimXpertFeatureType_e  
    Dim features As Object  
    Dim appliedFeatures As Object  
    Dim appliedAnnotations As Object  
    Dim appliedAnnotation As IDimXpertAnnotation  
    Dim feature As IDimXpertFeature  
    Dim appliedFeature As IDimXpertFeature  
    Dim msgStr As String  
    Dim msgStr2 As String  
    Dim msgStr3 As String  
    Dim msgStr4 As String  
    Dim n As Long
```

```

Dim o As Long
Dim p As Long
Dim boolstatus As Boolean
Sub main()
    swModel = swApp.ActiveDoc
    swModelDocExt = swModel.Extension
    swSelMgr = swModel.SelectionManager
    ' Get the default DimXpert schema using IModelDocExtension.DimXpertManager()
    swSchema = swModelDocExt.DimXpertManager("Default", True)
    ' Get IDimXpertPart from the IDimXpertManager
    swDXPart = swSchema.DimXpertPart
    Dim featCount As Long
    featCount = swDXPart.GetFeatureCount
    msgStr = "Total of "
    msgStr2 = featCount
    msgStr = msgStr + msgStr2 + " features in " + (swSchema.SchemaName)
    Debug.Print("")
    Debug.Print(msgStr)
    ' Get IDimXpert features through IDimXpertPart
    features = swDXPart.GetFeatures
    msgStr = (swSchema.SchemaName) + " has the following features: "
    Debug.Print("")
    Debug.Print(msgStr)
    For n = 0 To UBound(features)
        feature = features(n)
        Debug.Print(" " + "Feature name: " + (feature.Name))
        featureType = feature.Type
        Call GetPatternType(featureType, msgStr2)
        msgStr = "    Feature type "
        msgStr3 = " is suppressed on the DimXpertManager tab? "
        msgStr4 = feature.IsSuppressed()
        Debug.Print(msgStr + msgStr2 + msgStr3 + msgStr4)
        msgStr = "    " + "Model feature: "
        swFeature = feature.GetModelFeature()
        If Not (swFeature Is Nothing) Then
            msgStr2 = swFeature.GetTypeNames2()
            Debug.Print(msgStr + msgStr2)
        End If
        msgStr = "    " + "Number of SOLIDWORKS face entities in this feature: "
        msgStr2 = feature.GetFaceCount
        Debug.Print(msgStr + msgStr2)
        msgStr = "    " + "Number of applied features: "
        msgStr2 = feature.GetAppliedFeatureCount()
        Debug.Print(msgStr + msgStr2)
        appliedFeatures = feature.GetAppliedFeatures()
        If Not (IsNothing(appliedFeatures)) Then
            For o = 0 To UBound(appliedFeatures)
                appliedFeature = appliedFeatures(o)
                Debug.Print("        " + "Applied feature name: " + (appliedFeature.Name))
            Next
        End If
        msgStr = "    " + "Number of applied annotations: "
        msgStr2 = feature.GetAppliedAnnotationCount()
        Debug.Print(msgStr + msgStr2)
    Next
End Sub

```

```

appliedAnnotations = feature.GetAppliedAnnotations()
If Not (IsNothing(appliedAnnotations)) Then
    For p = 0 To UBound(appliedAnnotations)
        appliedAnnotation = appliedAnnotations(p)
        Debug.Print("    " + "Applied annotation name: " + (appliedAnnotation.Name))
    Next
End If
Debug.Print(" ")
Next

' If you know the name of a DimXpert feature, you can get it directly using IDimXpertPart.GetFeature("name"),
' which can return a general IDimXpertFeature or a more specific interface on the feature
' Get IDimXpertConeFeature for the Cone1 feature
Dim coneFeature As IDimXpertConeFeature
coneFeature = swDXPart.GetFeature("Cone1")
msgStr = coneFeature.Name + " is a DimXpert feature"
Debug.Print("")
Debug.Print(msgStr)
Debug.Print("")
' Get the nominal top plane coordinates for Cone1
Dim x As Double
Dim y As Double
Dim z As Double
Dim i As Double
Dim j As Double
Dim k As Double
Dim angle As Double
Debug.Print("Nominal top plane of Cone1")
Debug.Print("")
boolstatus = coneFeature.GetNominalTopPlane(x, y, z, i, j, k)
msgStr = "X-coordinate is "
msgStr2 = x
Debug.Print(msgStr + msgStr2)
msgStr = "Y-coordinate is "
msgStr2 = y
Debug.Print(msgStr + msgStr2)
msgStr = "Z-coordinate is "
msgStr2 = z
Debug.Print(msgStr + msgStr2)
msgStr = "I-component of unit vector is "
msgStr2 = i
Debug.Print(msgStr + msgStr2)
msgStr = "J-component of unit vector is "
msgStr2 = j
Debug.Print(msgStr + msgStr2)
msgStr = "K-component of unit vector is "
msgStr2 = k
Debug.Print(msgStr + msgStr2)
Debug.Print("")
' Get the nominal bottom plane coordinates for Cone1
Debug.Print("Nominal bottom plane of Cone1")
Debug.Print("")
boolstatus = coneFeature.GetNominalBottomPlane(x, y, z, i, j, k)
msgStr = "X-coordinate is "
msgStr2 = x

```

```

Debug.Print(msgStr + msgStr2)
msgStr = "Y-coordinate is "
msgStr2 = y
Debug.Print(msgStr + msgStr2)
msgStr = "Z-coordinate is "
msgStr2 = z
Debug.Print(msgStr + msgStr2)
msgStr = "I-component of unit vector is "
msgStr2 = i
Debug.Print(msgStr + msgStr2)
msgStr = "J-component of unit vector is "
msgStr2 = j
Debug.Print(msgStr + msgStr2)
msgStr = "K-component of unit vector is "
msgStr2 = k
Debug.Print(msgStr + msgStr2)
Debug.Print("")
' Get the nominal cone angle and coordinates for Cone1
Debug.Print("Nominal cone of Cone1")
Debug.Print("")
boolstatus = coneFeature.GetNominalCone(angle, x, y, z, i, j, k)
msgStr = "Angle is "
msgStr2 = (angle / 3.14159265358979) * 180
msgStr3 = " degrees"
Debug.Print(msgStr + msgStr2 + msgStr3)
msgStr = "X-coordinate is "
msgStr2 = x
Debug.Print(msgStr + msgStr2)
msgStr = "Y-coordinate is "
msgStr2 = y
Debug.Print(msgStr + msgStr2)
msgStr = "Z-coordinate is "
msgStr2 = z
Debug.Print(msgStr + msgStr2)
msgStr = "I-component of unit vector is "
msgStr2 = i
Debug.Print(msgStr + msgStr2)
msgStr = "J-component of unit vector is "
msgStr2 = j
Debug.Print(msgStr + msgStr2)
msgStr = "K-component of unit vector is "
msgStr2 = k
Debug.Print(msgStr + msgStr2)
Debug.Print("")
' Is the cone feature for a hole or a pin?
boolstatus = coneFeature.Inner
msgStr = "The cone is a hole and not a pin: "
msgStr2 = boolstatus
Debug.Print(msgStr + msgStr2)
End Sub

Public Sub GetPatternType(ByRef featureType As swDimXpertFeatureType_e, ByRef msgStr2 As String)
    If (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Plane) Then
        msgStr2 = "Plane"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Cylinder) Then

```

```

    msgStr2 = "Cylinder"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Cone) Then
    msgStr2 = "Cone"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Extrude) Then
    msgStr2 = "Boss"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Fillet) Then
    msgStr2 = "Fillet"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Chamfer) Then
    msgStr2 = "Chamfer"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundHole) Then
    msgStr2 = "CompoundHole"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundWidth) Then
    msgStr2 = "CompoundWidth"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundNotch) Then
    msgStr2 = "CompoundNotch"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundClosedSlot3D) Then
    msgStr2 = "CompoundClosedSlot3D"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectPoint) Then
    msgStr2 = "IntersectPoint"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectLine) Then
    msgStr2 = "IntersectLine"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectCircle) Then
    msgStr2 = "IntersectCircle"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectPlane) Then
    msgStr2 = "IntersectPlane"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Pattern) Then
    msgStr2 = "Pattern"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Sphere) Then
    msgStr2 = "Sphere"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_BestfitPlane) Then
    msgStr2 = "Bestfit plane"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Surface) Then
    msgStr2 = "Surface"
End If
End Sub
Public swApp As SldWorks
End Class

```

Get DimXpert Datum Example (VBA)

This example shows how to build a part and get attributes for the DimXpert Datum annotation.

```
' -----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\cosmosflopress\ball valve\ball.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars.  
' 3. Click the Add DimXpert datum icon on the DimXpert toolbar.  
' 4. Click the ball of the part.  
' 5. Click to place the datum annotation.  
' 6. Click the green check mark to accept the new datum feature.  
' 7. Observe the following DimXpert features on the DimXpertManager tab  
'    of the Management Panel: Sphere1.  
' 8. Open an Immediate window.  
' 9. Ensure that the SOLIDWORKS DimXpert type library is loaded  
'    in Tools > References.  
' 10. Ensure that the Microsoft Scripting Runtime library is loaded  
'     in Tools > References.  
'  
' Postconditions:  
' 1. Writes the output of this macro to c:\temp\dimXpertInfo.txt.  
' 2. Inspect the Immediate window.  
'  
' NOTE: Because the part is used elsewhere, do not save changes.  
' -----
```

Option Explicit

```
Dim strs As New Collection
```

```
Dim dimXpertPart As dimXpertPart
```

```
Sub Main()
```

```
    Dim swapp As SldWorks.SldWorks
```

```
    Set swapp = Application.SldWorks
```

```
    Dim swModelDoc As SldWorks.ModelDoc2
```

```
    Set swModelDoc = swapp.ActiveDoc
```

```
    If swModelDoc Is Nothing Then
```

```
        Exit Sub
```

```
    End If
```

```
    Dim f As New FileSystemObject
```

```
    Dim textStr As TextStream
```

```
    Set textStr = f.CreateTextFile("C:\temp\dimXpertInfo.txt", True)
```

```
    If textStr Is Nothing Then
```

```
        Debug.Print "Error creating temp file."
```

```
        Exit Sub
```

```
    End If
```

```

    Call log("Starting DimXpert log...", textStr)
    Call retrieve_info_text(swapp, textStr)
    textStr.Close
End Sub
Private Sub log(text As String, textStr As TextStream)
    Debug.Print text
    textStr.WriteLine (text)
End Sub
Private Sub retrieve_info_text(swapp As SldWorks.SldWorks, textStr As TextStream)
    Dim dimXpertMgr As SldWorks.DimXpertManager

    Set dimXpertMgr =
swapp.IActiveDoc2.Extension.DimXpertManager(swapp.IActiveDoc2.IGetActiveConfiguration().Name,
True)
    Call log("Model: " & swapp.IActiveDoc2.GetPathName, textStr)
    Dim dimXpertPartObj As dimXpertPart
    Set dimXpertPartObj = dimXpertMgr.dimXpertPart

    Set dimXpertPart = dimXpertPartObj

    Dim vAnnotations As Variant

    vAnnotations = dimXpertPart.GetAnnotations()

    Call log("-----", textStr)
    Call log("Annotations...", textStr)
    Call log("-----", textStr)

    Dim annotationTemp As DimXpertAnnotation
    Dim annotationIndex As Long
    For annotationIndex = 0 To UBound(vAnnotations)
        Set annotationTemp = vAnnotations(annotationIndex)

        Dim AnnotationDataText As Collection

        Set AnnotationDataText = AnnotationData(annotationTemp)
        Dim annotationTextIndex As Long
        For annotationTextIndex = 1 To AnnotationDataText.Count
            Call log(AnnotationDataText(annotationTextIndex), textStr)
        Next
    Next
End Sub

```

```

Public Function AnnotationData(annotation As DimXpertAnnotation) As Collection
    Dim annoType As Long

    'general info
    Call GeneralInfo(annotation)
    annoType = annotation.Type
    If annoType = swDimXpertDatum Then
        Call DatumData(annotation)

    ElseIf annoType = swDimXpertGeoTol_Position Then
        Call PositionData(annotation)

    ElseIf annoType = swDimXpertGeoTol_CompositePosition Then
        Call CompositePositionData(annotation)

    ElseIf annoType = swDimXpertGeoTol_Symmetry Then
        Call SymmetryData(annotation)

    ElseIf annoType = swDimXpertGeoTol_Concentricity Then
        Call ConcentricityData(annotation)

    ElseIf annoType = swDimXpertGeoTol_LineProfile Then
        Call LineProfileData(annotation)

    ElseIf annoType = swDimXpertGeoTol_CompositeLineProfile Then
        Call CompositeLineProfileData(annotation)

    ElseIf annoType = swDimXpertGeoTol_SurfaceProfile Then
        Call SurfaceProfileData(annotation)

    ElseIf annoType = swDimXpertGeoTol_CompositeSurfaceProfile Then
        Call CompositeSurfaceProfileData(annotation)

    ElseIf annoType = swDimXpertGeoTol_Angularity Or annoType = swDimXpertGeoTol_Parallelism Or
annoType = swDimXpertGeoTol_Perpendicularity Then
        Call OrientationData(annotation)

    ElseIf annoType = swDimXpertGeoTol_TotalRunout Then
        Call TotalRunoutData(annotation)

    ElseIf annoType = swDimXpertGeoTol_CircularRunout Then
        Call CircularRunoutData(annotation)

    ElseIf annoType = swDimXpertGeoTol_Flatness Then
        Call FlatnessData(annotation)

    ElseIf annoType = swDimXpertGeoTol_Circularity Then

```

```

        Call CircularityData(annotation)

    ElseIf annoType = swDimXpertGeoTol_Cylindricity Then
        Call CylindricityData(annotation)

    ElseIf annoType = swDimXpertGeoTol_Straightness Then
        Call StraightnessData(annotation)
    ElseIf annoType = swDimXpertGeoTol_Tangency Then
        Call TangencyData(annotation)
    Else ' any of the dimension tolerance types
        Call DimensionToleranceData(annotation)

    End If

    Set AnnotationData = strs
End Function

Private Sub Clear(strs As Collection)
    Dim n As Long
    strs.Remove (strs.Count)
    If Not strs.Count = 0 Then
        Call Clear(strs)
    End If
End Sub

Private Sub GeneralInfo(annotation As DimXpertAnnotation)
    Dim annoType As String
    Dim modelObj As Object
    Dim modelFeature As SldWorks.feature
    If Not strs.Count = 0 Then
        Call Clear(strs)
    End If

    strs.Add ("")
    strs.Add ("Name: " + annotation.Name)

    annoType = annotationTypeNameFromObject(annotation)
    strs.Add ("Type: " + annoType)
    strs.Add ("Display Entity: " + DisplayEntity(annotation))

    Set modelObj = annotation.GetModelFeature
    Set modelFeature = modelObj
    If Not (modelFeature Is Nothing) Then
        strs.Add ("ModelFeature: " + modelFeature.Name + " (" + modelFeature.GetType + " " + modelFeature.GetTypeName2() +
        ")")
    End If
End Sub

Private Sub DatumData(annotation As DimXpertDatum)

```

```

' the datum letter
strs.Add ("")
strs.Add ("Datum Letter:  " + annotation.Identifier)

```

End Sub

```
Private Sub PositionData(annotation As DimXpertPositionTolerance)
```

```

    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean, value As Double
    Dim boolstatus As Boolean

```

```

    strs.Add ("")
    strs.Add ("Position Tolerance Compartment:")

```

```

' the shape of the tolerance zone
strs.Add ("  Zone Type: " + PositionZoneType(annotation.zoneType))

```

```

' the zone vector if the tolerance zone is planar
If annotation.zoneType = swDimXpertPositionZoneType_PlanarPosition Then
    boolstatus = annotation.GetPlanarZoneVector(I, J, K)
    strs.Add ("  Direction: " + FormatVector(I, J, K))
End If

```

```

' the tolerance value
strs.Add ("  Tolerance: " + FormatDouble(annotation.Tolerance))

```

```

' the material condition modifier applied to feature
strs.Add ("  Modifier: " + mcmStr(annotation.modifier))

```

```

' the projected tolerance zone when specified
boolstatus = annotation.GetProjectedZone(enabled, value)
Call FormatProjectedZone(enabled, value)

```

```

' the datum reference frame
Call DatumsStr(annotation)

```

End Sub

```
Private Sub CompositePositionData(annotation As DimXpertCompositePositionTolerance)
```

```

    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean, value As Double
    Dim boolstatus As Boolean

```

```

    strs.Add ("")
    strs.Add ("Composite Position Tolerance Compartment")

```

```

' the shape of the tolerance zone

```

```

strs.Add (" Zone Type: " + PositionZoneType(annotation.zoneType))

' the zone vector when the zone is planar
If annotation.zoneType = swDimXpertPositionZoneType_PlanarPosition Then
    boolstatus = annotation.GetPlanarZoneVector(I, J, K)
    strs.Add (" Direction: " + FormatVector(I, J, K))
End If

' the tolerance value
strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

' the material condition modifier
strs.Add (" Modifier: " + mcmStr(annotation.modifier))

' the projected tolerance zone when specified
boolstatus = annotation.GetProjectedZone(enabled, value)
Call FormatProjectedZone(enabled, value)

' the datum reference frame for the pattern location
Call DatumsStr(annotation)

' the datum reference frame for the feature to feature location
strs.Add ("Composite datums:")
strs.Add (" Repeat Primary: " + IIf(annotation.PrimaryInLowerTier, "True", "False"))
strs.Add (" Repeat Secondary: " + IIf(annotation.SecondaryInLowerTier, "True", "False"))
strs.Add (" Repeat Tertiary: " + IIf(annotation.TertiaryInLowerTier, "True", "False"))

End Sub

Private Sub SymmetryData(annotation As DimXpertSymmetryTolerance)
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean
    strs.Add ("")
    strs.Add ("Symmetry Tolerance Compartment")

    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

    ' the material condition modifier applied to feature
    strs.Add (" Modifier: " + mcmStr(annotation.modifier))

    ' the datum reference frame
    strs.Add ("")
    Call DatumsStr(annotation)

    ' the direction of the planar zone
    strs.Add ("")
    boolstatus = annotation.GetZoneDirection(I, J, K)

```

```

        strs.Add ("Planar Zone Direction: " + FormatVector(I, J, K))

End Sub

Private Sub ConcentricityData(annotation As DimXpertConcentricityTolerance)

    strs.Add (""")
    strs.Add ("Concentricity Tolerance Compartment")

    ' the tolerance value
    strs.Add ("  Tolerance: " + FormatDouble(annotation.Tolerance))

    ' the material condition modifier applied to feature
    strs.Add ("  Modifier: " + mcmStr(annotation.modifier))

    ' the datum reference frame
    Call DatumsStr(annotation)

End Sub

Private Sub TotalRunoutData(annotation As DimXpertTolerance)
    strs.Add (""")
    strs.Add ("Total Runout Tolerance Compartment")

    ' the tolerance value
    strs.Add ("  Tolerance: " + FormatDouble(annotation.Tolerance))

    ' the datum reference frame
    Call DatumsStr(annotation)

End Sub

Private Sub CircularRunoutData(annotation As DimXpertTolerance)
    strs.Add (""")
    strs.Add ("Circular Runout Tolerance Compartment")

    ' the tolerance value
    strs.Add ("  Tolerance: " + FormatDouble(annotation.Tolerance))

    ' the datum reference frame
    Call DatumsStr(annotation)

End Sub

Private Sub LineProfileData(annotation As DimXpertLineProfileTolerance)
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean

    strs.Add (""")
    strs.Add ("Line Profile Tolerance Compartment")

```

```

' the tolerance value
strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

' the outer (outside material) tolerance value
strs.Add (" Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue))

' the vector normal to the profile zones
strs.Add ("")
boolstatus = annotation.GetPlanarZoneVector(I, J, K)
strs.Add ("Planar Zone Vector: " + FormatVector(I, J, K))

' the datum reference frame
Call DatumsStr(annotation)

```

End Sub

```
Private Sub CompositeLineProfileData(annotation As DimXpertCompositeLineProfileTolerance)
```

```

strs.Add ("")
strs.Add ("Composite Line Profile Tolerance Compartment")

' the tolerance value
strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

' the outer (outside material) tolerance value
strs.Add (" Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue))

' the vector normal to the profile zones
strs.Add ("")
boolstatus = annotation.GetPlanarZoneVector(I, J, K)
strs.Add ("Planar Zone Vector: " + FormatVector(I, J, K))

' the datum reference frame
Call DatumsStr(annotation)

' the datum reference frame for the orientation and form
strs.Add ("Composite Datums:")
strs.Add (" Repeat Primary: " + IIf(annotation.PrimaryInLowerTier, "True", "False"))
strs.Add (" Repeat Secondary: " + IIf(annotation.SecondaryInLowerTier, "True", "False"))
strs.Add (" Repeat Tertiary: " + IIf(annotation.TertiaryInLowerTier, "True", "False"))

```

End Sub

```
Private Sub SurfaceProfileData(annotation As DimXpertSurfaceProfileTolerance)
```

```

strs.Add ("")
strs.Add ("Surface Profile Tolerance Compartment")

```

```

' the tolerance value
strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

' the outer (outside material) tolerance value
strs.Add (" Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue))

' the datum reference frame
Call DatumsStr(annotation)

End Sub

Private Sub CompositeSurfaceProfileData(annotation As DimXpertCompositeSurfaceProfileTolerance)

strs.Add ("")
strs.Add ("Composite Surface Profile Tolerance Compartment")

' the tolerance value
strs.Add (" Tolerance Upper Tier: " + FormatDouble(annotation.Tolerance))

' the outer tolerance value
strs.Add (" Outer Tolerance Upper Tier: " + FormatDouble(annotation.OuterToleranceValue))

' the datum reference frame for the location
Call DatumsStr(annotation)

' the datum reference frame for the orientation and form
strs.Add ("Composite Datums:")
strs.Add (" Repeat Primary: " + IIf(annotation.PrimaryInLowerTier, "True", "False"))
strs.Add (" Repeat Secondary: " + IIf(annotation.SecondaryInLowerTier, "True", "False"))
strs.Add (" Repeat Tertiary: " + IIf(annotation.TertiaryInLowerTier, "True", "False"))
End Sub

Private Sub OrientationData(annotation As DimXpertOrientationTolerance)
Dim I As Double, J As Double, K As Double
Dim enabled As Boolean, value As Double
Dim annoType As Long
Dim boolstatus As Boolean
annoType = annotation.Type
' the type or orientation tolerance
If annoType = swDimXpertGeoTol_Perpendicularity Then
strs.Add ("Orientation Type: Perpendicularity")
ElseIf annoType = swDimXpertGeoTol_Parallelism Then
strs.Add ("Orientation Type: Parallelism")
ElseIf annoType = swDimXpertGeoTol_Angularity Then
strs.Add ("Orientation Type: Angularity")
End If

```

```

strs.Add ("")
strs.Add ("Orientation Tolerance Compartment:")

' the shape of the tolerance zone
Select Case annotation.zoneType
Case swDimXpertOrientationZoneType_Cylindrical
    strs.Add (" Zone Type: Cylindrical")
Case swDimXpertOrientationZoneType_Planar
    strs.Add (" Zone Type: Planar")
    boolstatus = annotation.GetPlanarZoneVector(I, J, K)
    strs.Add ("Planar Zone Vector: " + FormatVector(I, J, K))
End Select

' the tolerance value
strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

'material condition modifier applied to feature
strs.Add (" Modifier: " + mcmStr(annotation.modifier))

' the projected tolerance zone when specified
boolstatus = annotation.GetProjectedZone(enabled, value)
Call FormatProjectedZone(enabled, value)

' is tangent plane
strs.Add (" IsTangentPlane: " + IIf(annotation.IsTangentPlane, "True", "False"))

' the datum reference frame
Call DatumsStr(annotation)

```

End Sub

```

Private Sub FlatnessData(annotation As DimXpertFlatnessTolerance)
    Dim enabled As Boolean
    Dim length As Double, width As Double
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean

    strs.Add ("")
    strs.Add ("Flatness Tolerance Compartment:")

    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

    ' the per unit area data
    boolstatus = annotation.GetPerUnitArea(enabled, length, width, I, J, K)
    strs.Add (" Per Unit Area: " + IIf(enabled, "True", "False"))
    If enabled Then

```

```
        strs.Add (" Per Unit Length: " + FormatDouble(length))
        strs.Add (" Per Unit Width: " + FormatDouble(width))
        strs.Add (" Per Unit Direction: " + FormatVector(I, J, K))
    End If
```

End Sub

```
Private Sub CircularityData(annotation As DimXpertTolerance)
    strs.Add ("")
    strs.Add ("Circularity Tolerance Compartment")

    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
```

End Sub

```
Private Sub CylindricityData(annotation As DimXpertTolerance)
    strs.Add ("")
    strs.Add ("Cylindricity Tolerance Compartment")

    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
```

End Sub

```
Private Sub StraightnessData(annotation As DimXpertStraightnessTolerance)
    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean
    Dim dist As Double
    Dim boolstatus As Boolean
    'tolerance compartment info
    strs.Add ("")
    strs.Add ("Straightness Tolerance Compartment")

    'type or shape of the tolerance zone
    Select Case annotation.zoneType
    Case swDimXpertStraightnessZoneType_Cylindrical
        strs.Add (" Zone Type: Cylindrical")
    Case swDimXpertStraightnessZoneType_PlanarMedian
        strs.Add (" Zone Type: Planar Median")
    Case swDimXpertStraightnessZoneType_Surface
        strs.Add (" Zone Type: Surface Straightness")
        boolstatus = annotation.GetPlanarZoneVector(I, J, K)
        strs.Add (" Zone Vector: " + FormatVector(I, J, K))
    End Select

    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
```

```

'per unit length
boolstatus = annotation.GetPerUnitLength(enabled, dist)
strs.Add (" Per Unit Length: " + IIf(enabled, "True", "False"))
If enabled Then
    strs.Add (" Per Unit Length Distance: " + FormatDouble(dist))
End If

' the material condition modifier
strs.Add (" Modifier: " + mcmStr(annotation.modifier))
End Sub

Private Sub ProfileData(annotation As DimXpertTolerance)
    strs.Add ("")
    strs.Add ("Profile Tolerance Compartment")
    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub

Private Sub SurfaceFinishData(annotation As DimXpertTolerance)
    strs.Add ("")
    strs.Add ("Surface Finish Tolerance Compartment")
    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
End Sub

Private Sub TangencyData(annotation As DimXpertTolerance)
    strs.Add ("")
    strs.Add ("Tangency Tolerance Compartment")
    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
End Sub

Private Sub DimensionToleranceData(annotation As DimXpertDimensionTolerance)
    Dim isAngleType As Boolean
    Dim annoType As Long
    Dim upper As Double, lower As Double
    Dim plus As Double, minus As Double
    Dim boolstatus As Boolean
    annoType = annotation.Type

    isAngleType = False

    strs.Add ("")
    strs.Add ("Dimension Tolerance Compartment")
    If annoType = swDimXpertDimTol_DistanceBetween Then
        Dim distancebetween As DimXpertDistanceBetweenDimTol
        Set distancebetween = annotation
        Call DistanceBetweenData(distancebetween)
    End If
End Sub

```

```

ElseIf annoType = swDimXpertDimTol_CompositeDistanceBetween Then
    Dim compdistancebetween As DimXpertCompositeDistanceBetweenDimTol
    Set compdistancebetween = annotation
    Call CompositeDistanceBetweenData(compdistancebetween)

ElseIf annoType = swDimXpertDimTol_CounterBore Then
    Dim counterBore As IDimXpertCounterBoreDimTol
    Set counterBore = annotation
    If counterBore.ReferenceFeature Is Nothing Then
        strs.Add ("Reference Feature: NULL")
    Else
        strs.Add ("Reference Feature: " + counterBore.ReferenceFeature.Name)
    End If

ElseIf annoType = swDimXpertDimTol_Depth Then
    Dim depth As IDimXpertDepthDimTol
    Set depth = annotation
    If depth.ReferenceFeature Is Nothing Then
        strs.Add ("Reference Feature: NULL")
    Else
        strs.Add ("Reference Feature: " + depth.ReferenceFeature.Name)
    End If

ElseIf annoType = swDimXpertDimTol_CounterSinkDiameter Then
    Dim countersink As IDimXpertCounterSinkDiameterDimTol
    Set countersink = annotation
    If countersink.ReferenceFeature Is Nothing Then
        strs.Add ("Reference Feature: NULL")
    Else
        strs.Add ("Reference Feature: " + countersink.ReferenceFeature.Name)
    End If

ElseIf annoType = swDimXpertDimTol_ChamferDimension Then

    Select Case annotation.ChamferType
    Case swDimXpertChamferDimensionType_Angle
        strs.Add ("Chamfer Dimension Type: Angle")
        isAngleType = True
    Case swDimXpertChamferDimensionType_LinearDistance1
        strs.Add ("Chamfer Dimension Type: Distance 1")
    Case swDimXpertChamferDimensionType_LinearDistance2
        strs.Add ("Chamfer Dimension Type: Distance 2")
    End Select

```

```

ElseIf annoType = swDimXpertDimTol_AngleBetween Then
    isAngleType = True
    Dim angleBetween As IDimXpertAngleBetweenDimTol
    Set angleBetween = annotation
    ' the origin and tolerance feature
    strsr.Add ("Origin Feature: " + angleBetween.OriginFeature.Name)

    ' is supplement angle
    strsr.Add ("Supplement Angle: " + IIf(angleBetween.Supplement, "True", "False"))

ElseIf annoType = swDimXpertDimTol_CounterSinkAngle Then
    isAngleType = True
    Dim countersinkAngle As IDimXpertCounterSinkAngleDimTol
    Set countersinkAngle = annotation

    If countersinkAngle.ReferenceFeature Is Nothing Then
        strsr.Add ("Reference Feature: NULL")
    Else
        strsr.Add ("Reference Feature: " + countersinkAngle.ReferenceFeature.Name)
    End If

ElseIf annoType = swDimXpertDimTol_ConeAngle Then
    isAngleType = True
End If

' conversion for radians to degrees when dimension type is angle
Dim dbl As Double
If isAngleType Then
    dbl = 57.2957795130823
Else
    dbl = 1#
End If

' the nominal, and upper and lower limits of size of the dimension
strsr.Add ("")
strsr.Add ("Compute Nominal Dimension: " + FormatDouble(annotation.GetNominalValue * dbl))
boolstatus = annotation.GetUpperAndLowerLimit(upper, lower)
strsr.Add ("Get Upper Limit: " + FormatDouble(upper * dbl))
strsr.Add ("Get Lower Limit: " + FormatDouble(lower * dbl))

' the upper and lower tolerance value by type
Select Case annotation.DimensionType

Case swDimXpertDimTolType_BlockTolerance
    strsr.Add ("Dimension Type: Block Tolerance")

```

```

' block tolerance
Case swDimXpertDimTolType_BlockToleranceNoNominal
    Dim blockTols As DimXpertBlockTolerances
    Set blockTols = dimXpertPart.GetBlockTolerances

    Select Case blockTols.Type
        Case swDimXpertBlockToleranceType_ASMEInch
            str.Add ("Dimension Type: Block Tolerance No Nominal")
            If isAngleType Then
                str.Add ("Angular Block Tolerance")
            Else
                str.Add ("Block Tolerance Decimal Places: " +
Format(annotation.BlockToleranceDecimalPlaces, "##,##0"))
            End If
        Case swDimXpertBlockToleranceType_ISO2768
            str.Add ("Dimension Type: General Tolerance")
    End Select

Case swDimXpertDimTolType_ISOLimitsAndFits
    str.Add ("Dimension Type: Limits and Fits")
' limits and fits tolerance
Case swDimXpertDimTolType_ISOLimitsAndFitsNoNominal
    str.Add ("Dimension Type: Limits and Fits No Nominal")
    str.Add ("Limits and Fits: " + annotation.LimitsAndFitsCode)
' limit dimension
Case swDimXpertDimTolType_LimitDimension
    str.Add ("Dimension Type: Limit Dimension")
    boolstatus = annotation.GetUpperAndLowerLimit(upper, lower)
    str.Add ("Get Upper Limit: " + FormatDouble(upper * dbl))
    str.Add ("Get Lower Limit: " + FormatDouble(lower * dbl))
Case swDimXpertDimTolType_MAXTolerance
    str.Add ("Dimension Type: MAXTolerance")
Case swDimXpertDimTolType_MINTolerance
    str.Add ("Dimension Type: MINTolerance")
Case swDimXpertDimTolType_NoTolerance
    str.Add ("Dimension Type: NoTolerance")
Case swDimXpertDimTolType_PlusMinusDimension
    str.Add ("Dimension Type: Plus Minus Dimension")

' plus and minus dimension
Case swDimXpertDimTolType_PlusMinusNoNominal
    str.Add ("Dimension Type: Plus Minus No Nominal")
    boolstatus = annotation.GetPlusAndMinusTolerance(plus, minus)
    str.Add ("Plus Tolerance: " + FormatDouble(plus * dbl))
    str.Add ("Minus Tolerance: " + FormatDouble(minus * dbl))

End Select

```

```

End Sub

Private Sub DistanceBetweenData(annotation As DimXpertDistanceBetweenDimTol)
    Dim feature As DimXpertFeature
    Dim featureFosUsage As Long
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean
    ' the origin and tolerance feature along with their feature of size usage (min, max, center)
    boolstatus = annotation.GetOriginFeature(feature, featureFosUsage)
    str.Add ("")
    str.Add ("Origin Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))
    boolstatus = annotation.GetFeature(feature, featureFosUsage)
    str.Add ("Tolerance Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))

    ' The direction vector
    boolstatus = annotation.GetDirectionVector(I, J, K)
    str.Add ("")
    str.Add ("Direction Vector: " + FormatVector(I, J, K))

End Sub

Private Sub CompositeDistanceBetweenData(annotation As DimXpertCompositeDistanceBetweenDimTol)
    Dim feature As DimXpertFeature
    Dim featureFosUsage As Long
    Dim plus As Double, minus As Double
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean
    Dim blockTols As DimXpertBlockTolerances

    ' the origin and tolerance feature along with their feature of size usage (min, max, center)
    boolstatus = annotation.GetOriginFeature(feature, featureFosUsage)
    str.Add ("")
    str.Add ("Origin Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))
    boolstatus = annotation.GetFeature(feature, featureFosUsage)
    str.Add ("Tolerance Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))

    ' the pattern locating feature
    boolstatus = annotation.GetIntraFeature(feature, featureFosUsage)
    str.Add ("Pattern Locating Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))

    ' The direction vector
    boolstatus = annotation.GetDirectionVector(I, J, K)
    str.Add ("")
    str.Add ("Direction Vector: " + FormatVector(I, J, K))

    ' the upper and lower tolerance value for the pattern location by type

```

```

Select Case annotation.DimensionType
' plus and minus dimension
Case swDimXpertDimTolType_PlusMinusNoNominal
    strsr.Add ("Pattern Locating Dimension Type: Plus Minus No Nominal")
    boolstatus = annotation.GetPlusAndMinusToleranceIntraFeature(plus, minus)
    strsr.Add (" Plus Tolerance: " + FormatDouble(plus))
    strsr.Add (" Minus Tolerance: " + FormatDouble(minus))
' block tolerance
Case swDimXpertDimTolType_BlockToleranceNoNominal

    Set blockTols = dimXpertPart.GetBlockTolerances

    Select Case blockTols.Type
    Case swDimXpertBlockToleranceType_ASMEInch
        strsr.Add ("Pattern Locating Dimension Type: Block Tolerance No Nominal")
        strsr.Add (" Block Tolerance Decimal Places: " +
Format(annotation.BlockToleranceDecimalPlaces, "##,##0"))
    Case swDimXpertBlockToleranceType_ISO2768
        strsr.Add ("Pattern Locating Dimension Type: General Tolerance")
    End Select

End Select

End Select

' the upper and lower tolerance value for the feature to feature location by type
Select Case annotation.DimensionTypeIntraFeature
' plus and minus dimension
Case swDimXpertDimTolType_PlusMinusNoNominal
    strsr.Add ("Feature Locating Dimension Type: Plus Minus No Nominal")
    boolstatus = annotation.GetPlusAndMinusToleranceIntraFeature(plus, minus)
    strsr.Add (" Plus Tolerance: " + FormatDouble(plus))
    strsr.Add (" Minus Tolerance: " + FormatDouble(minus))
' block tolerance
Case swDimXpertDimTolType_BlockToleranceNoNominal

    Set blockTols = dimXpertPart.GetBlockTolerances
    Select Case blockTols.Type
    Case swDimXpertBlockToleranceType_ASMEInch
        strsr.Add ("Feature locating Dimension Type: Block Tolerance No Nominal")
        strsr.Add (" Block Tolerance Decimal Places: " +
Format(annotation.BlockToleranceDecimalPlacesIntraFeature, "##,##0"))
    Case swDimXpertBlockToleranceType_ISO2768
        strsr.Add ("Feature locating Dimension Type: General Tolerance")
    End Select

End Select

End Sub

```

```

Private Sub DatumsStr(annotation As DimXpertTolerance)
    strs.Add ("")
    strs.Add ("Datums:")
    Call datumStr(annotation.GetPrimaryDatums(), annotation.GetPrimaryDatumModifiers(), "
Primary:")
    Call datumStr(annotation.GetSecondaryDatums(), annotation.GetSecondaryDatumModifiers(), "
Secondary:")
    Call datumStr(annotation.GetTertiaryDatums(), annotation.GetTertiaryDatumModifiers(), "
Tertiary:")

End Sub

Private Sub datumStr(dats As Variant, mods As Variant, datumOrder As String)
    Dim I As Long
    Dim str As String
    Dim mcm As Long
    If IsEmpty(dats) Then
        str.Add (datumOrder + ": none")
        Exit Sub
    End If

    For I = LBound(dats) To UBound(dats)
        mcm = mods(I)
        str = str + " " + dats(I).Identifier + " @ " + mcmStr(mcm)
    Next I

    If StrComp(str, "") > 0 Then
        str.Add (datumOrder + str)
    Else
        str.Add (datumOrder + " <none>")
    End If
End Sub

' returns a string containing the height of the projected tolerance zone
Private Sub FormatProjectedZone(enabled As Boolean, height As Double)

    If enabled = True Then
        str.Add (" Projected Zone: True")
        str.Add (" Zone Height: " + FormatDouble(height))
    Else
        str.Add (" Projected Zone: False")
    End If

End Sub

End Sub

Private Function mcmStr(mcm As Long) As String
    Dim str

    Select Case mcm
    Case swDimXpertMCM_LMC
        str = "LMC"

```

```

Case swDimXpertMCM_MMC
    str = "MMC"
Case swDimXpertMCM_NoMCM
    str = "NoMCM"
Case swDimXpertMCM_RFS
    str = "RFS"
End Select

mcmStr = str
End Function
' returns a string containing the type of position tolerance zone used
Private Function PositionZoneType(typ As Long) As String
    Dim str As String

    Select Case typ
    Case swDimXpertPositionZoneType_CylindricalPosition
        str = "Cylindrical"
    Case swDimXpertPositionZoneType_PlanarPosition
        str = "Planar"
    Case swDimXpertPositionZoneType_SphericalPosition
        str = "Spherical"
    Case swDimXpertPositionZoneType_Boundary
        str = "Boundary"
    Case swDimXpertPositionZoneType_RadialPositionArc
        str = "RadialPositionArc"
    Case swDimXpertPositionZoneType_RadialPositionPlanar
        str = "RadialPositionPlanar"
    Case Else
        str = "N/A"
    End Select

    PositionZoneType = str
End Function
' returns a string containing the names of the SW display entities
Private Function DisplayEntity(annotation As DimXpertAnnotation) As String
    Dim str As String
    Dim dispEnt As Object
    Dim swAnnot As SldWorks.annotation

    'Set dispEnt = swDimXpert.GetDisplayEntity(annot)
    Set dispEnt = annotation.GetDisplayEntity

    If Not dispEnt Is Nothing Then
        If TypeOf dispEnt Is SldWorks.annotation Then
            Set swAnnot = dispEnt

```

```

        str = swAnnot.GetName
    End If
End If

DisplayEntity = str
End Function
' returns a string containing the feature of size usage (min, max, center)
Private Function FosUsage(value As Long) As String
    Dim str
    Select Case value
    Case swDimXpertDistanceFosUsage_Center
        str = "Center"
    Case swDimXpertDistanceFosUsage_MaximumSide
        str = "Max"
    Case swDimXpertDistanceFosUsage_MinimumSide
        str = "Min"
    Case Else
        str = "N/A"
    End Select
    FosUsage = str
End Function
Private Function FormatVector(I As Double, J As Double, K As Double) As String
    Dim str
    str = FormatDouble(I) + ", " + FormatDouble(J) + ", " + FormatDouble(K)

    FormatVector = str
End Function
Private Function FormatDouble(value As Double) As String
    Dim str

    str = Format(value, "##,##0.0000")

    FormatDouble = str
End Function
Private Function RadiansToDegrees(value As Double) As String
    Dim str

    str = Format((value * 57.2957795130823), "##,##0.00")

    RadiansToDegrees = str
End Function
Private Function annotationTypeNameFromObject(anno As DimXpertAnnotation) As String
    annotationTypeNameFromObject = annotationTypeNameFromTypeNumber(anno.Type)
End Function
Private Function annotationTypeNameFromTypeNumber(annoTypeIndex As Long) As String
    Select Case annoTypeIndex

```

```
Case swDimXpertDimTol_DistanceBetween
    annotationTypeNameFromTypeNumber = "DistanceBetween Dim"
Case swDimXpertDimTol_CounterBore
    annotationTypeNameFromTypeNumber = "CounterBore Dim"
Case swDimXpertDimTol_Depth
    annotationTypeNameFromTypeNumber = "Depth Dim"
Case swDimXpertDimTol_CounterSinkDiameter
    annotationTypeNameFromTypeNumber = "CounterSinkDiameter Dim"
Case swDimXpertDimTol_ChamferDimension
    annotationTypeNameFromTypeNumber = "ChamferDimension Dim"
Case swDimXpertDimTol_AngleBetween
    annotationTypeNameFromTypeNumber = "AngleBetween Dim"
Case swDimXpertDimTol_CounterSinkAngle
    annotationTypeNameFromTypeNumber = "CounterSinkAngle Dim"
Case swDimXpertDimTol_ConeAngle
    annotationTypeNameFromTypeNumber = "ConeAngle Dim"
Case swDimXpertDimTol_Diameter
    annotationTypeNameFromTypeNumber = "Diameter Dim"
Case swDimXpertDimTol_Length
    annotationTypeNameFromTypeNumber = "Length Dim"
Case swDimXpertDimTol_Radius
    annotationTypeNameFromTypeNumber = "Radius Dim"
Case swDimXpertDimTol_Width
    annotationTypeNameFromTypeNumber = "Width Dim"
Case swDimXpertDimTol_CompositeDistanceBetween
    annotationTypeNameFromTypeNumber = "CompositeDistanceBetween Dim"
Case swDimXpertDatum
    annotationTypeNameFromTypeNumber = "Datum"
Case swDimXpertGeoTol_Position
    annotationTypeNameFromTypeNumber = "Position Tol"
Case swDimXpertGeoTol_CompositePosition
    annotationTypeNameFromTypeNumber = "CompositePosition Tol"
Case swDimXpertGeoTol_Symmetry
    annotationTypeNameFromTypeNumber = "Symmetry Tol"
Case swDimXpertGeoTol_Concentricity
    annotationTypeNameFromTypeNumber = "Concentricity Tol"
Case swDimXpertGeoTol_LineProfile
    annotationTypeNameFromTypeNumber = "LineProfile Tol"
Case swDimXpertGeoTol_CompositeLineProfile
    annotationTypeNameFromTypeNumber = "CompositeLineProfile Tol"
Case swDimXpertGeoTol_SurfaceProfile
    annotationTypeNameFromTypeNumber = "SurfaceProfile Tol"
Case swDimXpertGeoTol_CompositeSurfaceProfile
    annotationTypeNameFromTypeNumber = "CompositeSurfaceProfile Tol"
Case swDimXpertGeoTol_Angularity
```

```

        annotationTypeNameFromTypeNumber = "Angularity Tol"
Case swDimXpertGeoTol_Parallelism
    annotationTypeNameFromTypeNumber = "Parallelism Tol"
Case swDimXpertGeoTol_Perpendicularity
    annotationTypeNameFromTypeNumber = "Perpendicularity Tol"
Case swDimXpertGeoTol_TotalRunout
    annotationTypeNameFromTypeNumber = "TotalRunout Tol"
Case swDimXpertGeoTol_CircularRunout
    annotationTypeNameFromTypeNumber = "CircularRunout Tol"
Case swDimXpertGeoTol_Flatness
    annotationTypeNameFromTypeNumber = "Flatness Tol"
Case swDimXpertGeoTol_Circularity
    annotationTypeNameFromTypeNumber = "Circularity Tol"
Case swDimXpertGeoTol_Cylindricity
    annotationTypeNameFromTypeNumber = "Cylindricity Tol"
Case swDimXpertGeoTol_Straightness
    annotationTypeNameFromTypeNumber = "Straightness Tol"
Case swDimXpertGeoTol_Tangency
    annotationTypeNameFromTypeNumber = "Tangency Tol"
Case Else
    annotationTypeNameFromTypeNumber = "<unknown> " & CStr(annoTypeIndex)

End Select
End Function

```